Deep Learning

Artificial intelligence

why I am starting this youtube channel?

Large neural Network? → youtube ∝

Advancements → AI → America

China → most of the research chinese

America → Awareness 1% GDP [AI infra]

India?

- AI Agents
- RAG
- RL
- LLM
- CUDA
- AWS/Azure/GCP/AI

∝ youtube

→ The things which I am learning why not teach them

[DSA] → ✓
[Webd] ✓
[Android] ✓

channel ✓

Find → People who are new to LLM world. ✓

Deep Learning

Prerequisites:-
- A) Maths (11th & 12th) ✓
- B) Any Programming Language (recommended Python) ✓

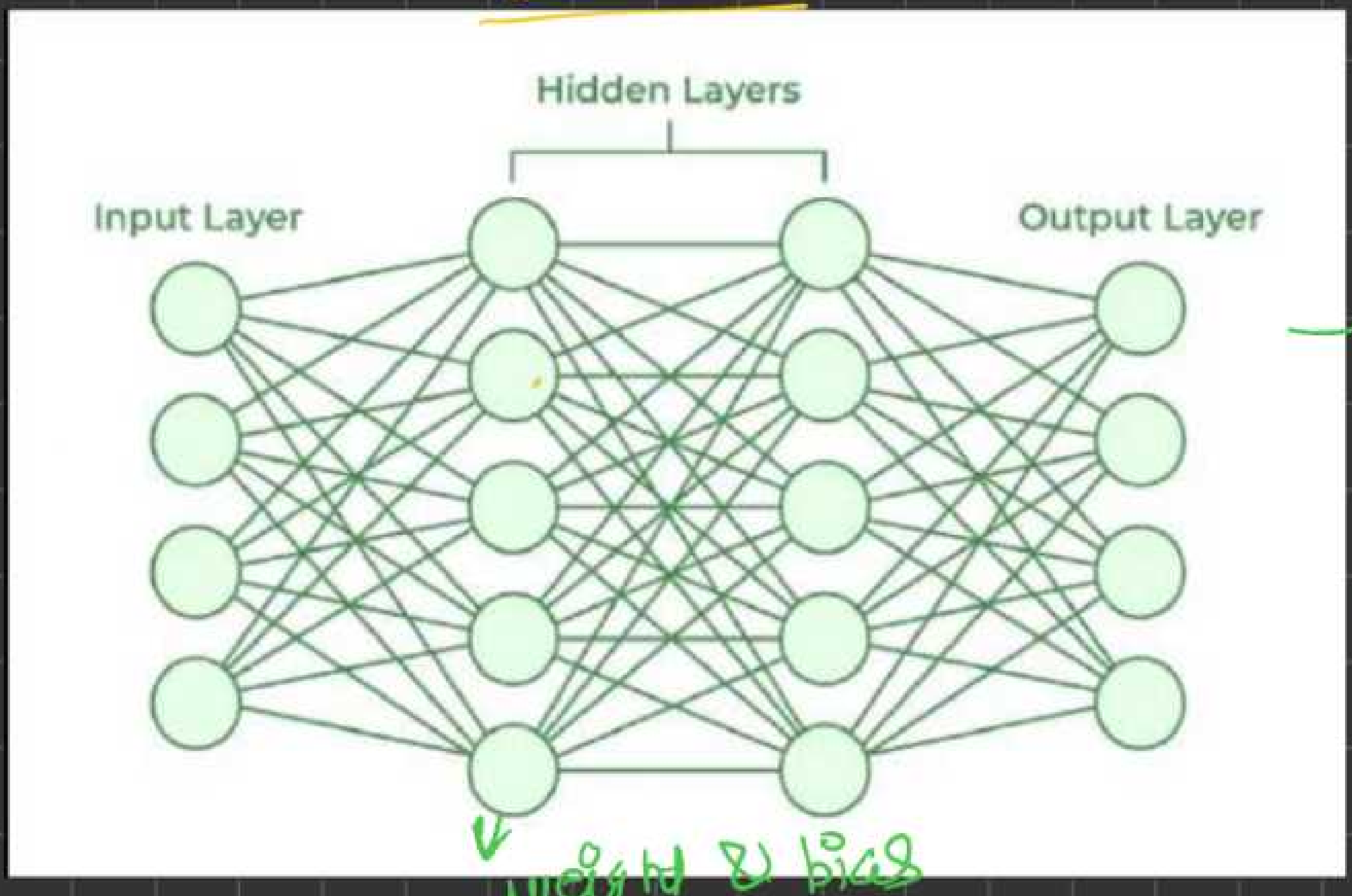
AI → Artificial intelligence ?

AI is the correlation and compression. ✓

Note:- America is giving 1% of GDP in AI infrastructure

$$\frac{1}{100} \times 30T = \boxed{0.3T}$$

Neural Networks



→ Neuron

← Backward Pass

→ forward Pass

weight & bias

Deep learning:- It is a subset of ML, that uses multi-layered artifical neural networks inspired by human brain.

Neural Networks are a special type of mathematical expressions.

Forward Pass & Backward Propagation

We will implement forward Pass & Backward Pass in Python. For learning purpose,
In production,
there are multiple libraries already implemented for neural network training.

Deep learning framework:-

1) $\boxed{\text{Pytorch}}$ 2) Tensorflow 3) Keras

1) Pytorch:- Meta, Indian (core) → Starting

→ $\boxed{\text{Proud}}$ → Vit

LeCunn

Vellore Indian

US → mtech

Somith Chintala

2) Tensorflow → developed by google (TPUs)

3) Keras → Francois chollet → 2015

4) JAX

In the simplest form, we want to write our own such DL library for education purpose. → microgad

---

microgad →

1) Forward Pass
2) Backward Pass
3) scalar values
4) suff. for binary classification.

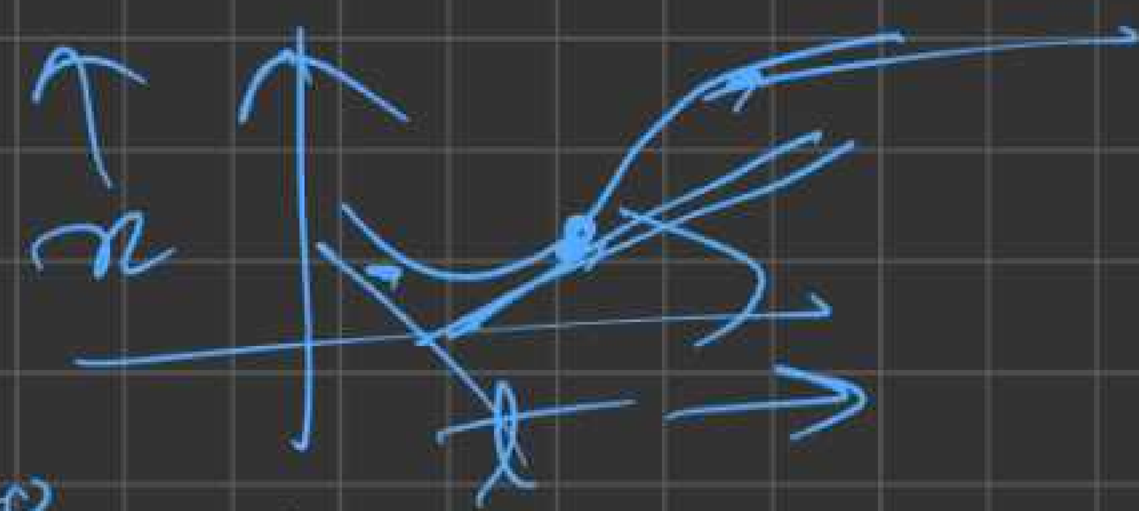
Differentiation

$v = \boxed{x/t}$ km/hr

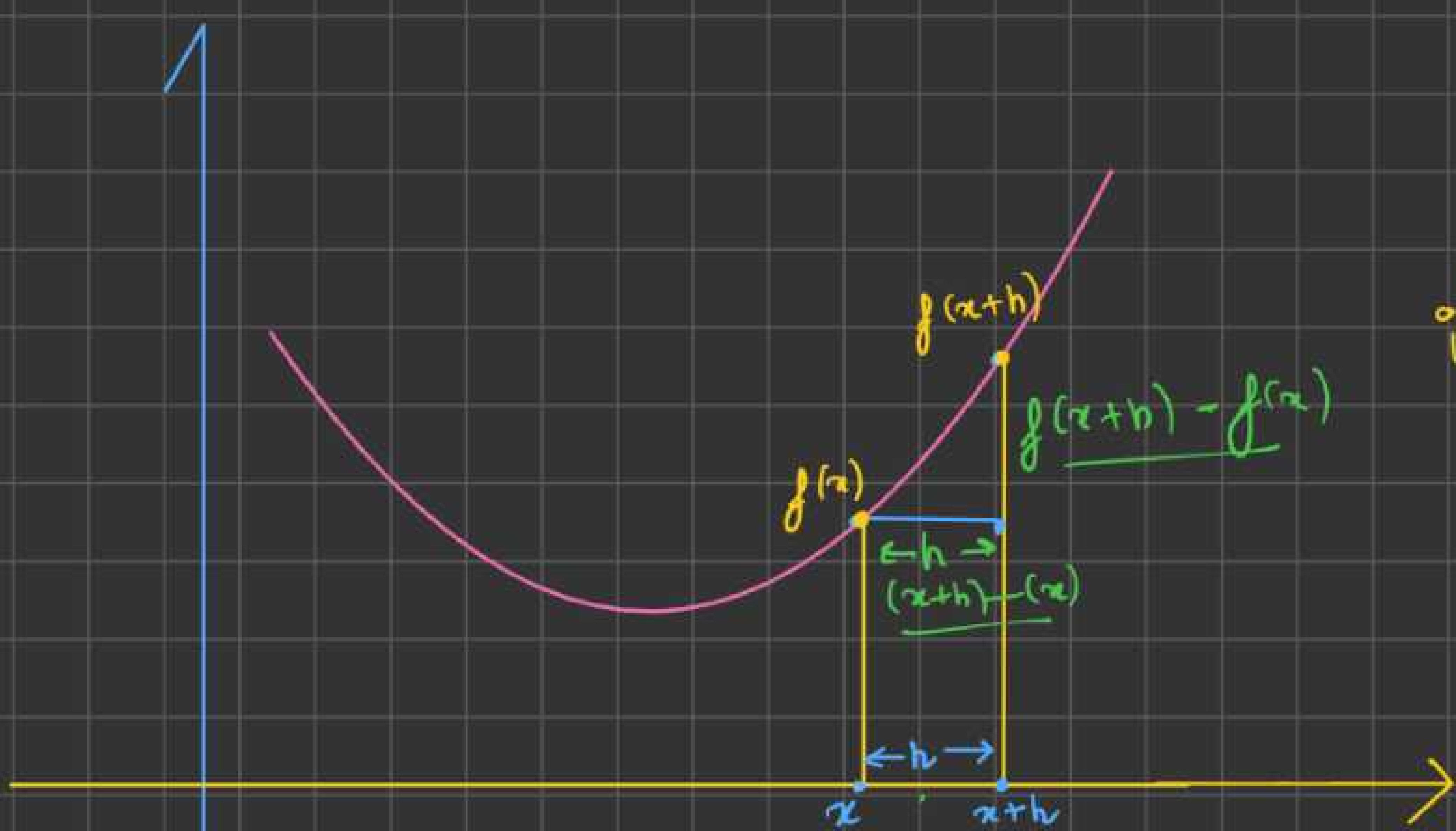
$dv = dx/dt$

→ v value

time terms



Function



$f(x) = x^2 + 2$

$x = 1,$

$f(x) = 3 = y$

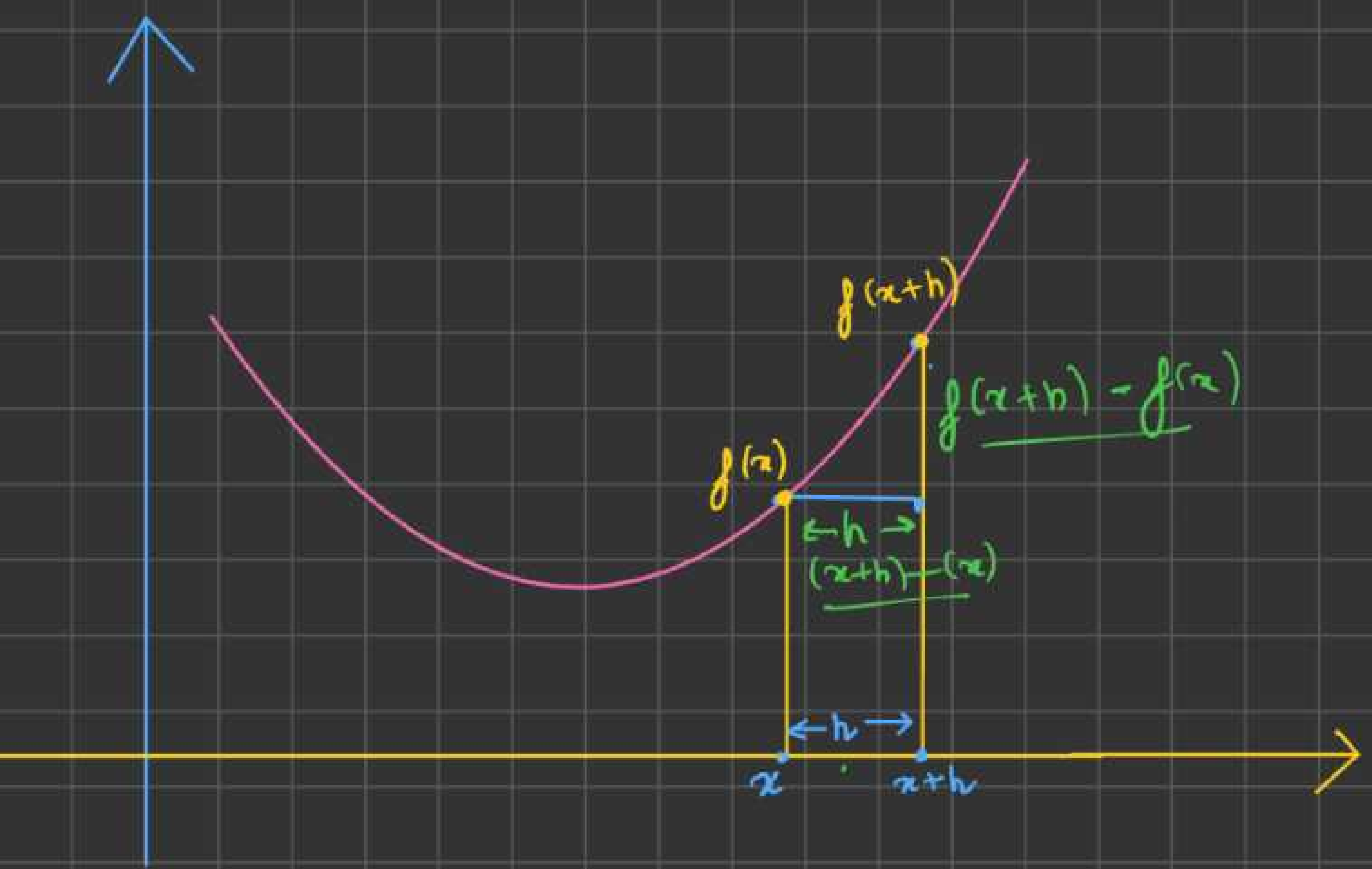
$$f'(x) = \frac{f(x+h) - f(x)}{(x+h) - (x)} = \boxed{\frac{f(x+h) - f(x)}{h} = f'(x)}$$

(slope)

(Assuming h is very small → 0)

We will see, this slope analytically & mathematically.

$\Rightarrow$



condition 1.-

i) slope > 0.

$x \to (x+h)$

which will be greater

$f(x+h)\uparrow$ or $f(x)\uparrow$

$f(x+h) > f(x)$

By Defn slope is the sensitivity of the measure of change of funn w.r.t. input.

The slope of above funn is +ve. ✓

so if h is added to x, then funn will definety increase.

Analytical observation

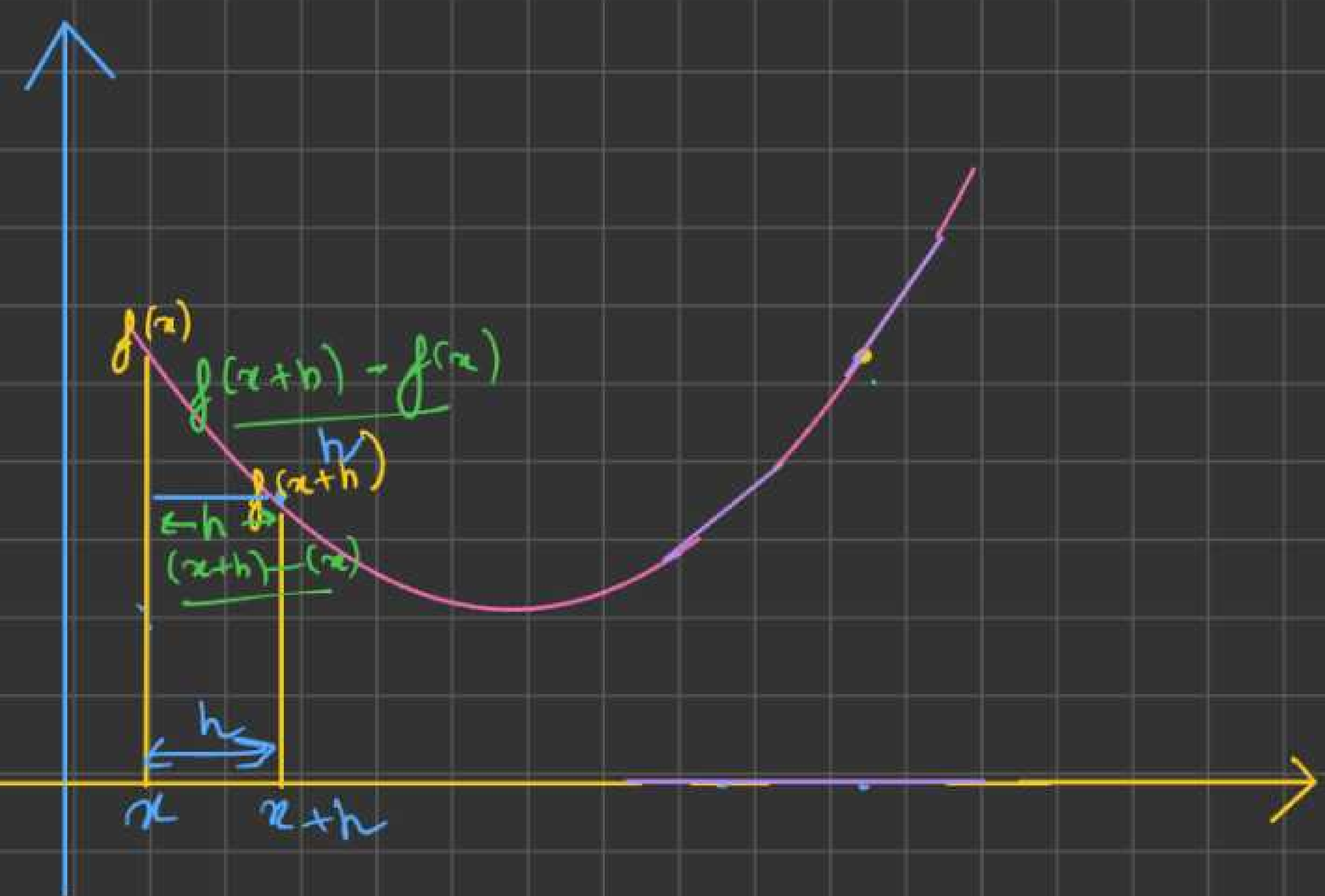
$y = f(x) \to$ Polynomial

$x^n = \boxed{nx^{n-1}} \to \frac{dy}{dx}$

Calculus

$\Rightarrow$

Made with Goodnotes



condition 2:-

i) slope < 0

$x \to (x+h)$

which will be greater

$f(x+h)\uparrow$ or $f(x)\uparrow$

$f(x+h) < f(x)$

By Def^n slope is the sensitivity of the measure of change of fun^n w.r.t. input.

$\boxed{\text{The slope of above fun}^n \text{ is -ve. So if h is added to x, then fun}^n \text{ will definitely decrease}}$

Analytical observation

$y = f(x) \to$ Polynomial

$x^n = \boxed{nx^{n-1}} \to \frac{dy}{dx}$

Calculus

x, x.grad, f(x)

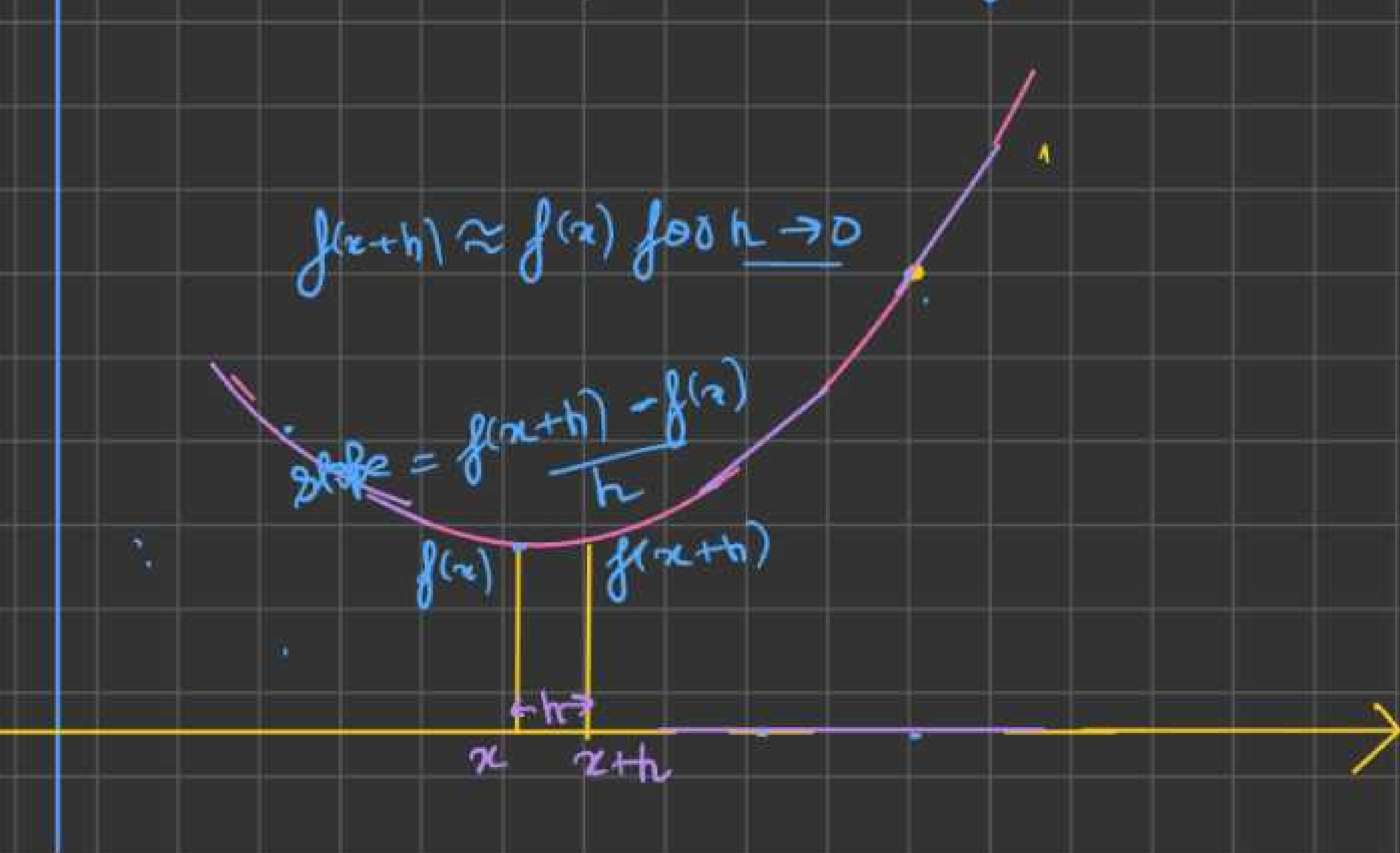
x.grad > 0, $f(x+h)\uparrow$

x.grad < 0 $f(x+h)\downarrow$

Analytical

$x+h$, x.grad $= 0$, $\therefore f(x)$ same.

condition 3



1) slope $= 0$:-

$x \to (x+h)$

which will be greater

$f(x+h) \uparrow$ or $f(x) \uparrow$

$f(x+h) \approx f(x)$

By Def^n slope is the sensitivity of the measure of change of fun^n w.r.t. input.

The slope of above fun^n is 0.
so if h is added to x, then fun^n will definetly remain same.

Analytical observation

$y = f(x) \to$ Polynomial
$x^n = \boxed{nx^{n-1}} \to \frac{dy}{dx}$

Calculus

x.grad ≈ 0
$x \to 0$, $f(x)$ remains same

$a, b, c \quad d = \underline{a * b + c}$

$a.grad = \frac{dd}{da}, \quad b.grad = \frac{dd}{db}, \quad c.grad = \frac{dd}{dc}$

Assume, $a = 2, \; b = -3, \; c = 10$ ✓

I $\underline{a = ?}$ $a.grad = ?$

slope $= \frac{f(x+h) - f(x)}{h} = \frac{f(a+h) - f(a)}{h}$

$= \frac{((a+h) * b + c) - (a * b + c)}{h} = \frac{\cancel{a*b} + h*b \cancel{+c} - \cancel{a*b} \cancel{-c}}{h} \approx \boxed{b}$

$\boxed{a.grad = b}$ (Pen & Paper)

$d = a * b + c, \quad a.grad = \frac{dd}{da} = \frac{d}{da}(a * b + c)$

$= \frac{d}{da}(a * b) + \cancel{\frac{dc}{da}}^{\,0}$ (As, $c = k$, $\underline{b = k2}$)

$= b\frac{da}{da} + a\cancel{\frac{db}{da}}^{\,0} = b, \quad \boxed{a.grad = b}$ (calculus)

---

we know that, ~~as~~ $b = -3$, so, $a.grad = -3$

$\boxed{a.grad < 0, \quad a > 0, \; d > 0}$

To decrease, d to 0, $\underline{\text{increase } a}$, as grad < 0

Made with Goodnotes

Assume, $a = 2$, $b = -3$, $c = 10$ ✓

$d = a * \underline{b} + c$

II $b = ?$ $b.grad = ?$

slope $= \dfrac{f(x+h) - f(x)}{h} = \dfrac{f(b+h) - f(b)}{h}$

$\dfrac{((a*(b+h)) + c) - (a*b+c)}{h} = \dfrac{\cancel{a*b} + a*h \cancel{+c} - \cancel{a*b} \cancel{-c}}{h}$

$= a$ $\boxed{b.grad = a}$ (maths defn)

$d = a*b+c$, $\dfrac{dd}{db} = \dfrac{d}{db}(a*b+c) = \dfrac{d}{db}(a*b) + \dfrac{dc}{db}$

$(c = k,\ a = k2)$

$= \left(a\dfrac{\cancel{db}}{\cancel{db}}\right) + \left(b * \cancel{\dfrac{da}{db}}^{\,0}\right) + \left(\cancel{\dfrac{dc}{db}}^{\,0}\right)$

$= a$, $\boxed{b.grad = a}$ (by calculus)

---

we know that, $a = 2$, ≥ 0, $b.grad = 2$

$\boxed{grad > 0,\ b < 0,\ d > 0}$

To decrease, d to 0, decrease $\underline{b}$, as grad > 0

Assume, $a=2$, $b=-3$, $c=10$ ✓

$d = a*b+c$

II $c = ?$ $c.grad = ?$

slope $= \frac{f(x+h)-f(x)}{h} = \frac{f(c+h)-f(c)}{h}$

$= \frac{(a*b+(c+h)) - (a*b+c)}{h} = \frac{\cancel{a*b} + \cancel{c} + h - \cancel{a*b} - \cancel{c}}{\cancel{h}}$

$= \boxed{1}$

$\boxed{c.grad = 1}$ (math's def^n)

$d = a*b+c$, $\frac{dd}{dc} = \frac{d}{dc}(a*b+c) = \frac{d}{dc}(a*b) + \frac{dc}{dc}$

$= \cancel{\frac{d(a*b)}{dc}}^{0} + \cancel{\frac{dc}{dc}}^{1} = 1$ $(a=k, b=k)$

$\boxed{c.grad = 1}$ (by calculus)

---

we know that, ~~as~~ $c=10$, so $c.grad = 1$

$\boxed{grad > 0, \; c > 0, \; d > 0}$

To decrease, d to 0, decrease c, as grad > 0

---

conclusion $d = a*b+c$

$a.grad = \frac{dd}{da} = b$, $b.grad = \frac{dd}{db} = a$, $c.grad = \frac{dd}{dc} =$

Code

Python → Basic OOPS

classes

Blue Point

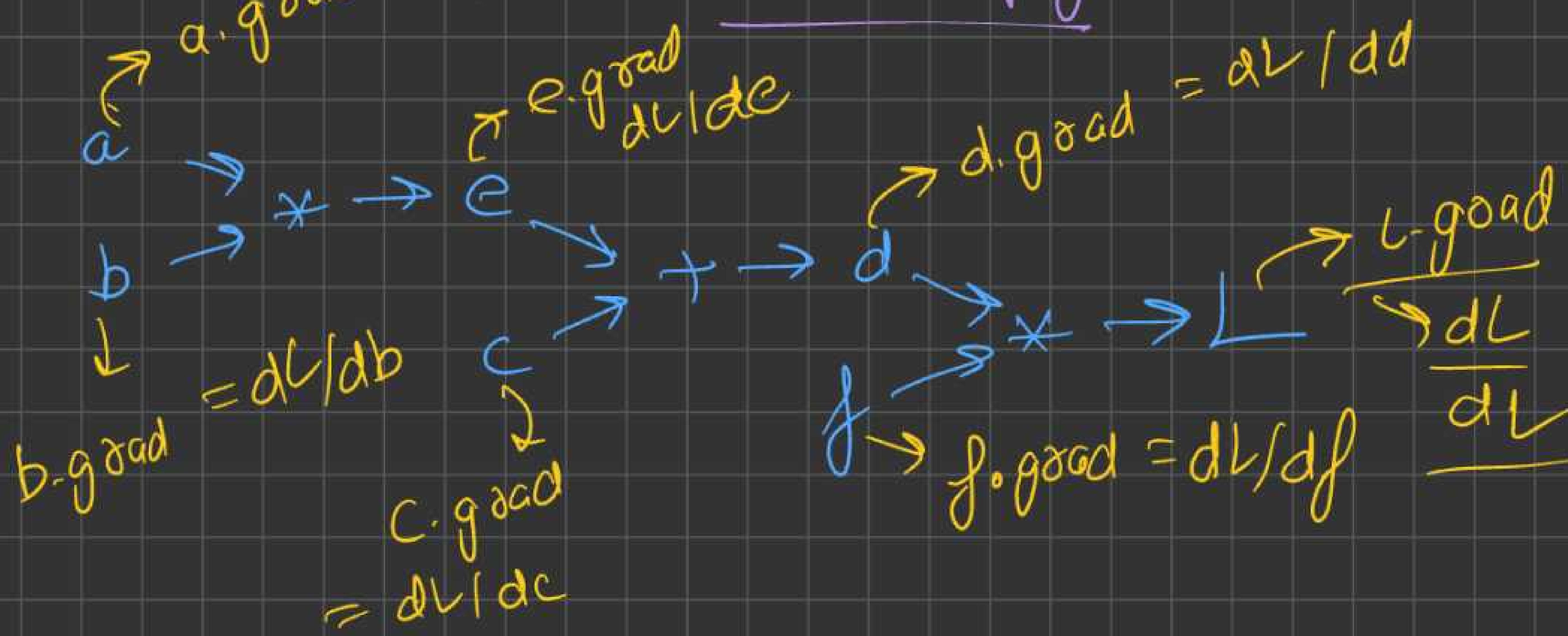
int, float,

bool, List

→ object

complex

Backward Propogation



---

why we set grad = 0 initially?

when slope of a $f(x)$ w.r.t. x is 0, at a certain pt., it means on changing x by tiny amt $(x \to x+h)$, $(x \to x-h)$, $f(x)$ remain same. Eg:- $dL/da = 0$, L is not changing on changing a by h, $(a+h)$ $(a-h)$.

A) L.grad = ? $\quad dL/dL$

$$\text{slope} = \frac{f(x+h) - f(x)}{h} = \frac{(\cancel{L}+h) - \cancel{L}}{h} = 1$$

$\boxed{\text{slope} = 1}$ → Analytical

B) $\dfrac{dL}{dL} = \dfrac{dL}{dL} = 1$ (How does L changes w.r.t. L)

$\boxed{\text{slope} = 1}$ → calculus

Made with Goodnotes

2) d.grad = ? $dL/dd = ?$, $L = d*f$

A) slope = $\frac{f(x+h)-f(x)}{h} = \frac{(d+h)*f - d*f}{h}$

$= \frac{\cancel{d*f} + \cancel{h}*f - \cancel{d*f}}{\cancel{h}} = \boxed{f}$ $\boxed{\text{slope} = f}$ → Analytical

B) $dL/dd = \frac{d(d*f)}{dd} = d\cancel{\frac{df}{dd}}^{0} + f\cancel{\frac{dd}{dd}}^{1}$ $(f = x = -2)$

$\boxed{\text{slope} = f}$ → calculus

$\frac{dL}{df}$ = f.grad = ? $L = d*f$

$\frac{dL}{df}$ = slope = $\frac{f(x+h)-f(x)}{h} = \frac{d(f+h) - df}{h}$

$= \frac{\cancel{df} + d\cancel{h} - \cancel{df}}{\cancel{h}} = d$, $\boxed{\text{slope} = d}$ → Analytically

B) $\frac{dL}{df} = \frac{d}{df}(d*f) = d\cancel{\frac{df}{df}} + f\cancel{\left(\frac{dd}{df}\right)}^{0}$ $(d = x = 4)$

$= d$

$\boxed{\text{slope} = d}$ → calculus

Made with Goodnotes

$c.grad = ?\quad \frac{dL}{dc} = ?$ $\quad L = d * f$

$L = (c+e) * f$

(introducing chain rule in calculus)

$A = 2B$
$B = 4C$
$A = 8C$

Basic understanding

$$\boxed{\frac{dz}{dx} = \left(\frac{dz}{dy}\right) * \left(\frac{dy}{dx}\right) \Rightarrow \frac{dc}{da} = \left(\frac{dc}{db}\right) * \left(\frac{db}{da}\right)}$$

chain rule in calculus

(Heart of the Back Prop)

(Heart of the NN)

$c.grad = \frac{dL}{dc}$, $c.grad = ?$

we know,
$d.grad = dL/dd$

$$\boxed{\frac{dL}{dc} = \frac{dL}{dd} * \frac{dd}{dc}} \Rightarrow \boxed{c.grad = d.grad * \frac{dd}{dc}}$$

$$c.grad = \underline{d.grad} * \frac{dd}{dc}$$

A) $\frac{dd}{dc} = ?$, $d = (c+e)$, $slope = \frac{f(x+h) - f(x)}{h}$

$\frac{dd}{dc} = ?$, $d = (c+e)$, $slope = \frac{f(x+h) - f(x)}{h}$

$slope = \frac{\cancel{(c+h+e)} - \cancel{(c+e)}}{\cancel{h}} = 1$,

$\boxed{\frac{dd}{dc} = 1}$

$c.grad = d.grad \times \cancel{\frac{dd}{dc}}$ = $\boxed{\begin{matrix} c.grad = d.grad \\ \frac{dL}{dc} = \frac{dL}{dd} \end{matrix}}$

Analytically

B) $\frac{dd}{dc} = ?$ $d = (c+e)$

$(e = k)$

$\frac{dd}{dc} = \frac{d}{dc}(c+e) = \cancel{\frac{dc}{dc}}^{1} + \cancel{\frac{de}{dc}}^{0}$

$\boxed{dd/dc = 1}$ → calculus

$c.grad = d.grad \times \left(\cancel{\frac{dd}{dc}}^{1}\right)$

$\boxed{\begin{matrix} c.grad = d.grad \\ dL/dc = dL/dd \end{matrix}}$ → calculus

Note:- when there is add^n, the gradients flows without changing

e.grad = ?, dL/de = ? $(d = c+e)$

we know that, dL/dd

with chain rule in calculus,

$$\boxed{\frac{dL}{de} = \frac{dL}{dd} * \frac{dd}{de}} \Rightarrow \text{e.grad} = \text{d.grad} * \frac{dd}{de}$$

dd/de = ?, $d = (c+e)$

A) $$\frac{dd}{de} = \text{slope} = \frac{f(x+h) - f(x)}{h} = \frac{(\cancel{c} + \cancel{e} + h) - (\cancel{c} + \cancel{e})}{\cancel{h}}$$

$$\boxed{\frac{dd}{de} = 1} \qquad \text{e.grad} = \text{d.grad} * \cancel{\frac{dd}{de}} \to 1$$

$$\boxed{\text{e.grad} = \text{d.grad}} \to \text{Analytical}$$

B) $\frac{dd}{de}$ = ?, $d = (c+e)$

$$\frac{dd}{de} = \frac{d}{de}(c+e) = \cancel{\frac{dc}{de}} \to 0 + \cancel{\frac{de}{de}} \to 1 \quad (c = k)$$

$$\boxed{dd/de = 1} \to \text{calculus} \qquad \boxed{\text{e.grad} = \text{d.grad}}$$

Made with Goodnotes

a.grad = ? dL/da = ?   dL/de = ✓, e = a * b

by chain rule,

$$\boxed{\frac{dL}{da} = \left(\frac{dL}{de}\right) * \left(\frac{de}{da}\right)} \rightarrow \text{chain rule.}$$

$$\frac{de}{da} = ?$$

A) de/da = ?, e = a * b, slope = $\frac{f(x+h) - f(x)}{h}$

$$\text{slope} = \frac{(a+h) * b - (a * b)}{h} = \frac{\cancel{ab} + \cancel{h}b - \cancel{ab}}{\cancel{h}} = b$$

$$\boxed{\frac{de}{da} = b}$$

$$\frac{dL}{da} = \frac{dL}{de} * \frac{de}{da}$$

$$a.grad = e.grad * \frac{de}{da}$$

$$\boxed{a.grad = b * (e.grad)}$$

→ Analytically

B) $\frac{de}{da} = ?$, e = a * b   (b = k)

$$\frac{d}{da}(a * b) = a\frac{db}{da} + b\frac{da}{da} = b$$

$$\boxed{de/da = b}$$

→ calculus

$$\boxed{a.grad = b * e.grad}$$

$\frac{dL}{db} = ?$, $\frac{dL}{de}$, $e = a * b$

Chain rule, $\boxed{\frac{dL}{db} = \frac{dL}{de} * \frac{de}{db}}$

A) $de/db = ?$, $e = a * b$, slope $= \frac{f(x+h) - f(x)}{h}$

slope $= \frac{(a * (b+h)) - (a * b)}{h} = \frac{\cancel{ab} + \cancel{h}a - \cancel{ab}}{\cancel{h}} = a$

$\boxed{de/db = a}$

$b.grad = e.grad * \frac{de}{db}$

Analytical $\longrightarrow$ $\boxed{b.grad = e.grad * a}$

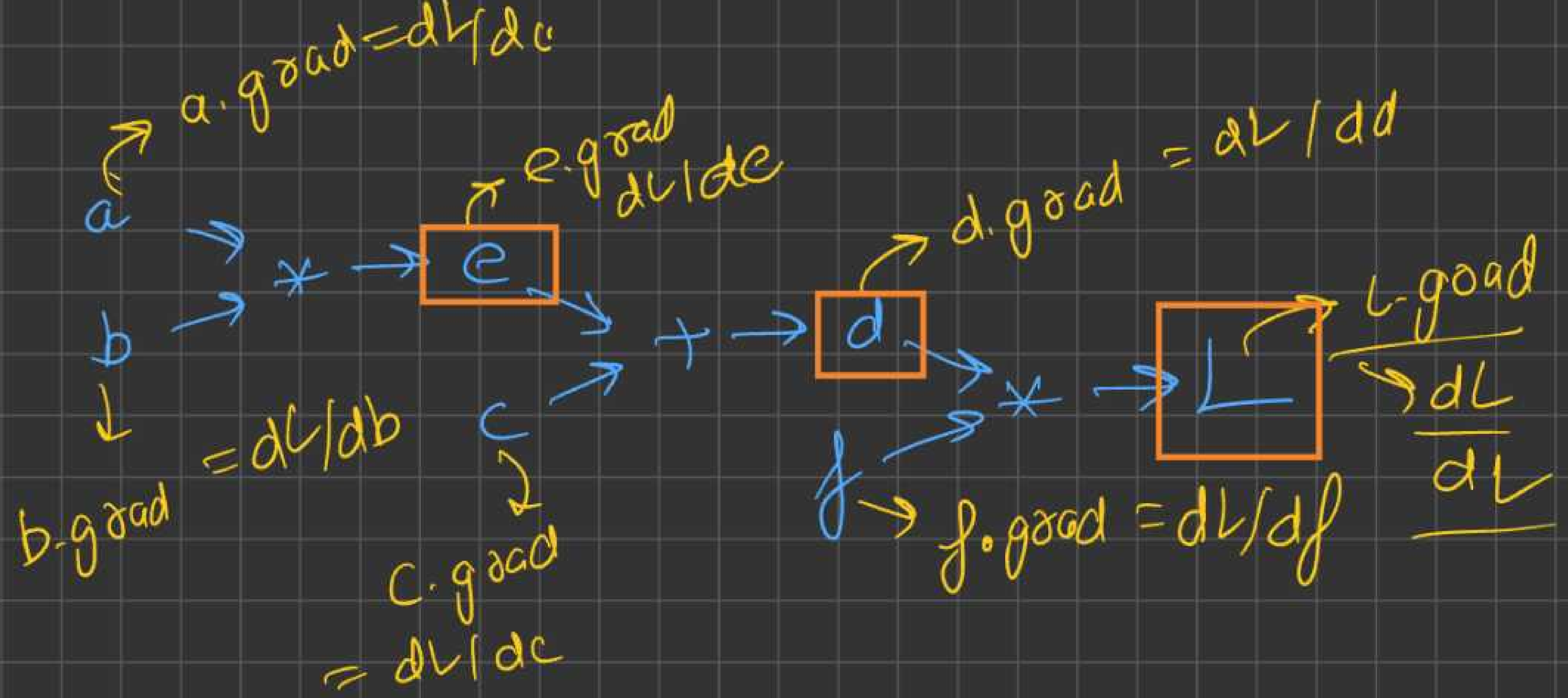
B) $\frac{de}{\partial b} = ?$ $e = a * b$

$(a = k)$

$\frac{d(a * b)}{db} = a\frac{db}{db} + b\frac{da}{db} = a$, $\boxed{\frac{de}{dB} = a}$

$\boxed{b.grad = e.grad * a}$ $\rightarrow$ calculus

Made with Goodnotes



$e = a*b$, $d = c+e$, $L = d*f$.

Forward Pass

$e = a*b$

$d = c+e$

$L = d*f$

(will we be updating intermediate nodes, like, e, d, L, as we have calc. grad for them) ?

Answer:- we calculated grad for all var. , a, b, c, d, e, f, L. , but we will only update, a, b, c, f. , but not e, d, L, bcz, e, d, L are intermed.

are intermediate steps, if we even ~~later~~ update, e, d, L, it won't make any sense, we will waste ops. if $e = a * b$, & we have updated, a & b, then even if we update e, it won't matter, as forward pass, we will overwrite, the value of e,

if $a \Rightarrow a+h$, $b \Rightarrow b-h$, $\boxed{e \rightarrow e+h'}$ → won't make sense

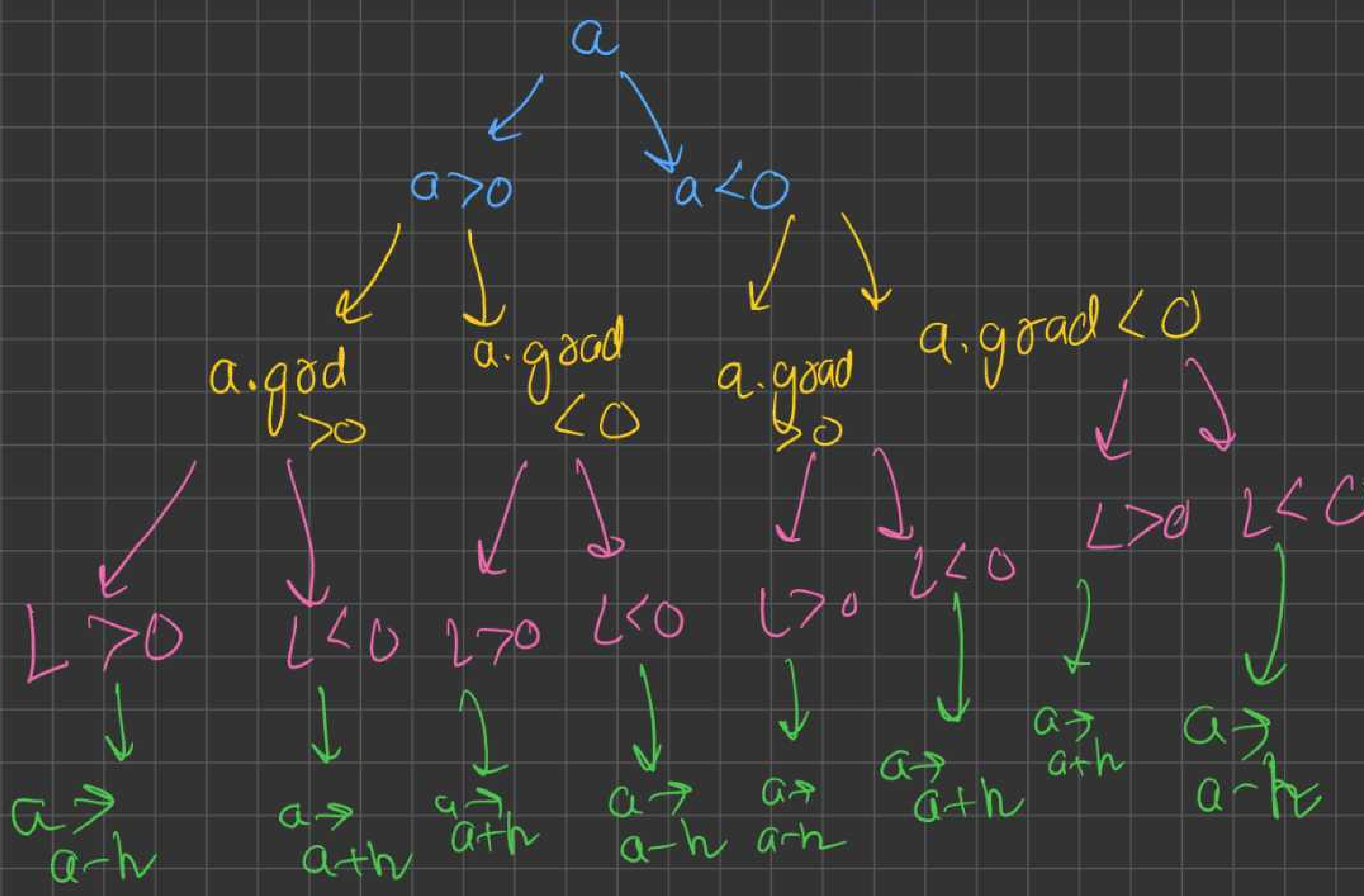
$$\boxed{e = (a+h)(b-h)}$$

The value of $e+h'$, will be overwritten by $(a+h)(b-h)$

↳ conclusion:- we will update only trainable parameters, not the intermediate parameters Eg (a, b, c, f)

general, Assume a variable 'a' is there

⇒



If ignoring, $a>0$, or $a<0$, (Assume, $L<0$)

if $a.grad > 0$, then, $\rightarrow a = a+h$

$a.grad < 0$, then, $\rightarrow$ ~~oo~~ $a = a-h$

$a \mathrel{+}= \alpha\, a.grad$ (Assume $h = a.grad$)

$a = a + (\alpha . (a.grad))$ ($a.grad > 0, < 0$)

$a = a + (\alpha . (a.grad)$ ($a.grad > 0$)

$a = a - (\alpha . (a.grad$ ($a.grad < 0$)

$\rightarrow \alpha = 0.00)$ ($h = \alpha . (a.grad)$

if $L<0$, conclusion, update parameters in dirn of gradient.

if $L>0$, conclusion, update parameter in opp. dirn of grad.

if $L<0$

a.data += α (a.grad)

α → learning rate

if $L>0$

a.data -= α (a.grad)

α → lear. rate

Implementing value class

other funⁿ

A) c = value(2.0) + value(3.0) = value(5.0)

this is correct

c = value(2.0) + 3.0 ⇒ this will give error.

bcz, value.add(float), but add funⁿ in value class expect both args to be value object

here, self = value, other = float

so, we have to convert float to a value object. (see code)

$$c = \underbrace{2.0}_{\text{float}} + \underbrace{\text{value}(3.0)}_{\text{value}}$$

$$c = \underbrace{\text{float}}_{\text{self}}.\text{add}(\underbrace{\text{value}}_{\text{other}})$$

(so, we will get error, we cannot sum float & value)

To resolve this issue, radd dunder method.

$$c = \underbrace{2.0}_{\text{self}} + \underbrace{\text{Value}(3.0)}_{\text{other}}$$

float. add (value)

So, use radd.

$$\Rightarrow \underbrace{\text{Value}}_{\text{self}}.\text{radd}(\underbrace{\text{float}}_{\text{other}})$$

(we know that, $a+b = b+a$)

(see code)

---

for __add__ we have (+) as a symbol, for __radd__ do we have any other symbol?

No, if add fun^n not work, then radd is automatically called.

$\Rightarrow$

Similarly multiplication will be exactly similar to add.

A) c = value(2.0) * value(3.0)

c = value(6.0) ✓ working

(value.mul(value)

B) c = value(2.0) * 3.0 (error)

(value.mult(float))

(error will be float has no object data)

to correct it we need to convert float → value.

C) c = 2.0 + value(3.0)

float.mul(value) (err. cannot

self other

sum float & value)

value.rmul(float)

self other

(but we know that, a*b = b*a)

(see code)

a) = a = Value (2.0) (this is correct)

a = -Value (2.0) (this will give err, as we have not implement __neg__)

a = -Value (2.0)

Value.neg(()) $\rightarrow$ self.

-Value(2.0) $\approx$ value (2.0) $*$ (-1)

$\Downarrow$

Value.mul(-1)

this is doable.

pow = ?

c = (Value(2.0)) ** 3 (this will give error)

__pow__ → dunder funⁿ

pow → int / float

Value. pow (int/float)

(Value(2.0)) . pow (3)

(Value(2.0)) ** 3

3 → ** 3 → Value(8.0, (2.0,), (**3))

Value(2.0) →

(*) sub = ? → $2-3 = -1$
$2+(-3) = -1$ these 2 are same thing

A) Value(2) - value(3) → $\approx$ value(2) + (- value(3))
self other

Value. sub(value)

value(2) + (-value(3))
↓
value. neg()
↓
Value. mul (-1) → int
↓
value(2) + value(-3) ⇒ value(-1)

So, children of value(2) - value(3)
⇓
$\boxed{((2,-3), '+')}$
→ for understanding

B) $\underbrace{\text{value}(2.0)}_{\text{value}} - \underbrace{3.0}_{\text{float}} \Rightarrow \underbrace{\text{value}}_{\text{self}}.\text{sub}(\underbrace{\text{float}}_{\text{other}})$

$\underbrace{\text{value}(2.0)}_{\text{self}} + \underbrace{(-3.0)}_{\text{other}}$

$\text{value}.\text{add}(\underbrace{\text{float}})$ → conv. float → value

⇒ $\boxed{\text{value}(-1.0),\ (2, -3),\ (+)}$

c) $\underbrace{2.0}_{\text{self}} - \text{value}\underbrace{(3.0)}_{\text{other}} \Rightarrow \text{float}.\text{sub}(\underline{\text{value}})$

(err, as float do not have any sub funⁿ)

$3.0 \leftarrow \underbrace{\text{Value}}_{\text{self}}.\text{rsub}(\underbrace{\text{float}}_{\text{other}}) \rightarrow 2.0$

```
def rsub(self, other):
    return other + (- self)
```

$\underbrace{\text{other}}_{\text{float}}$ + $(-\underbrace{\text{self}}_{\text{value.}})$

⇒ 2.0 - value(3.0)

⇒ float.sub(value) err

⇒ $\underbrace{\text{value}}_{\text{self}}$.rsub(float) ⇒ other + (-self)

3.0 ← self

other

2.0

2.0 3.0

float value

⇒ float.add(value.neg()) → err

⇒ value.mul(float) → -1

⇒ value(-3.0) ⇒ (prev = (3, -1), *)

⇒ float.add(value) → err

⇒ $\underbrace{\text{value}}_{\text{self}}$.radd($\underbrace{\text{float}}_{\text{other}}$) → 2.0

⇒ value(-3.0) + value(2.0)

⇒ $\boxed{\text{value}(-1.0)}$ ⇒ (prev = (-3, 2), +)

Made with Goodnotes

$a/b \Rightarrow a * \left(\frac{1}{b}\right) = a * (b^{-1}) = \underline{a * (b^{**-1})}$

(A) $\Rightarrow \frac{\text{value}(2.0)}{\text{value}(3.0)} = \text{value}(2) * \underline{(\text{value}(3.0)^{**-1})}$

$\Rightarrow (\text{value}(3.0))^{**-1}$

$\Rightarrow \boxed{\text{value}(0.34)}\ ((3.0,), **-1)$

$\Rightarrow \text{value}(2) * \text{value}(0.34)$

$\Rightarrow \boxed{\text{value}(0.68)}\ ((2, 0.34)), (*))$

B) $\Rightarrow \frac{\text{value}(2.0)}{3.0} = \text{value.div}(\text{float})$

2 self   other

$= \text{self} * (\text{other}^{**-1})$

$= \text{value}(2.0) * (3^{-1})$

$= \text{value}(2.0) * (0.34)$

$= \text{value.mul}(\text{float}) \rightarrow$ convert to value

$= \text{value}(0.68) \rightarrow ((2, 0.34)), (*)$

$\frac{2.0}{value(3.0)}$ ⇒ $\underset{\substack{\downarrow \\ self}}{float}$.div(value) (err, other float do not have div)

$\underset{self}{Value}$. rdiv ($\underset{\to other}{float}$)

= self * (other^{-1})

= value(3.0) * (2^{-1})

this is wrong

⇒ other * (self ** −1)

⇒ Value.pow (−1) ⇒ value (0.34)

(3.0)(** −1)

⇒ other * Value (0.34)

⇒ float.mul (Value) (err.)

⇒ $\underset{0.34}{value}$. rmul (float) → 2.0

= 0.34 * 2.0 = 0.68 (2, 0.34)

*

Activation Fun^n

Drawback - if Activation fun^n are not there, then, entire neural Network graph will be just a linear rel^{n}ship.

Key role of Activation fun^n is to introduce Non-linearity in your rel^{n}ships.

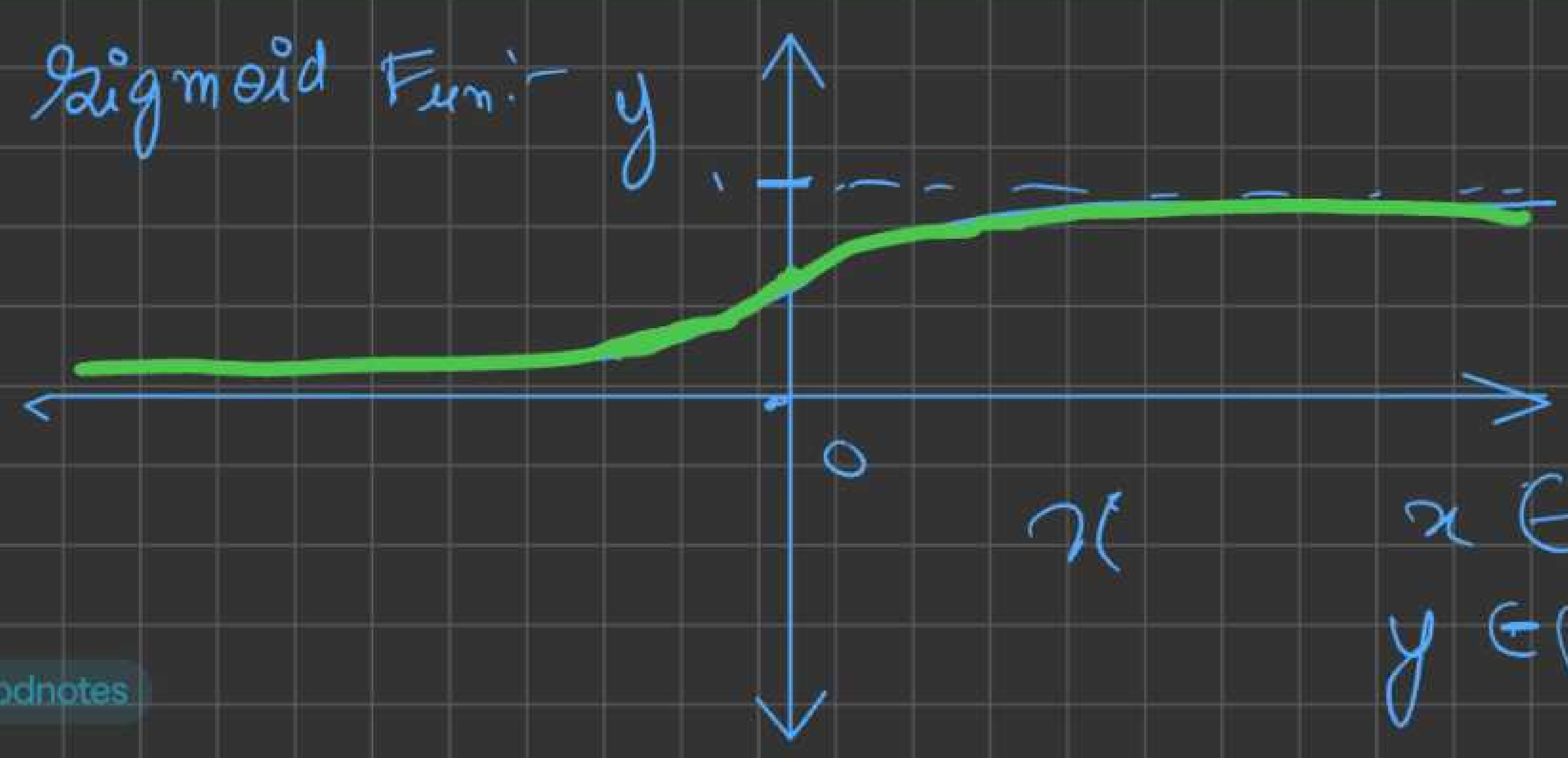
3 Most used Activation fun^n:-

1) Sigmoid = $\dfrac{1}{1+e^{-z}}$

2) tanh = $\dfrac{e^{z}-e^{-z}}{e^{z}+e^{-z}}$

3) relu = $\max(0, z)$

1) Sigmoid Fun:-



$x \in R$

$y \in (0,1)$

Derivation

we need to prove,

$$\sigma(z) = \frac{1}{1+e^{-x}} \quad ??$$

Probability = $\frac{\text{\# of outcomes for a event}}{\text{total \# of outcomes}}$

(How likely is the event)

$H = 20, T = 30$, $P(H) = 20/50$

$P(T) = 30/50$

$P(H) = 1 - P(T)$

$H = 50, T = 50$

Q How much more likely Head will come compare to Tail?

A ⇒ $\frac{H}{T} = \frac{50}{50} = \underline{1}$

if Tail comes once, then head also will occur once.

Equally likely H&T to occur.

$H = 80, T = 20$

Q How much more likely head will come from Tail?

Ans $\frac{H}{T} = \frac{80}{20} = 4$

if Tail comes once, then H will come 4x more

Q How much more like Tail will come from Head?

A = $\frac{T}{H} = \frac{20}{80} = \frac{1}{4} = 0.25$

if H comes, 4x times, then Tail comes 1x time, or,
if H comes 1x time, then tail comes 0.25x times

So, in conclusion, there is a event which is happening & which is not happening

$$\boxed{\text{odds} = H/T \ \& \ T/H}$$

$$\boxed{\text{Odds} = \frac{\text{\# of time of event happening}}{\text{\# of time of event not happening}}}$$

(Dividing by total outcome)

$$\text{Odds} = \frac{P(\text{event happening})}{P(\text{event not happening})}$$

$$\boxed{\text{Odds} = \frac{P}{1-P}}$$ ☆☆

$P(0,1)$

$P=0$, odds $=0$

$P=1$, odds $=\infty$

$(0,1) \xrightarrow{} (0,\infty)$

map

understanding log(x) significance?

$x \in (0, \infty)$ $\quad$ $y \in (-\infty, \infty)$

even for $10^{32} \rightarrow$ No. $\rightarrow$ log $\rightarrow 32$

$10^{-32} \rightarrow$ No. $\rightarrow$ log $\rightarrow -32$

So, log is introduced for squashing in neural network

$$\text{odds} = \frac{p}{1-p}$$

(taking log on both side)

$$\log\left(\frac{p}{1-p}\right) = z \qquad \frac{1}{1+e^{-z}} = f(z)$$

$$\frac{p}{1-p} = e^{z}$$

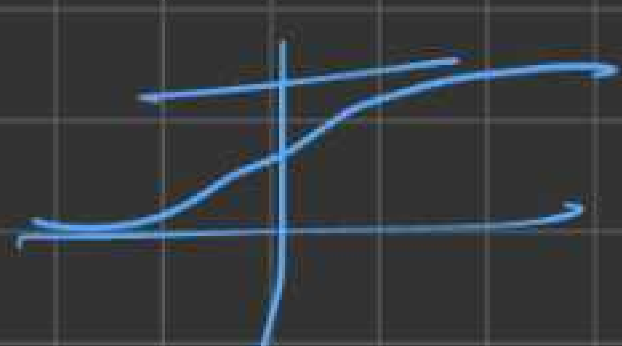
$$\frac{p}{1-p} = e^{z} \Rightarrow p = (1-p) \cdot e^{z}$$

$$p = e^{z} - pe^{z} \Rightarrow p = \frac{e^{z}}{1+e^{z}}$$

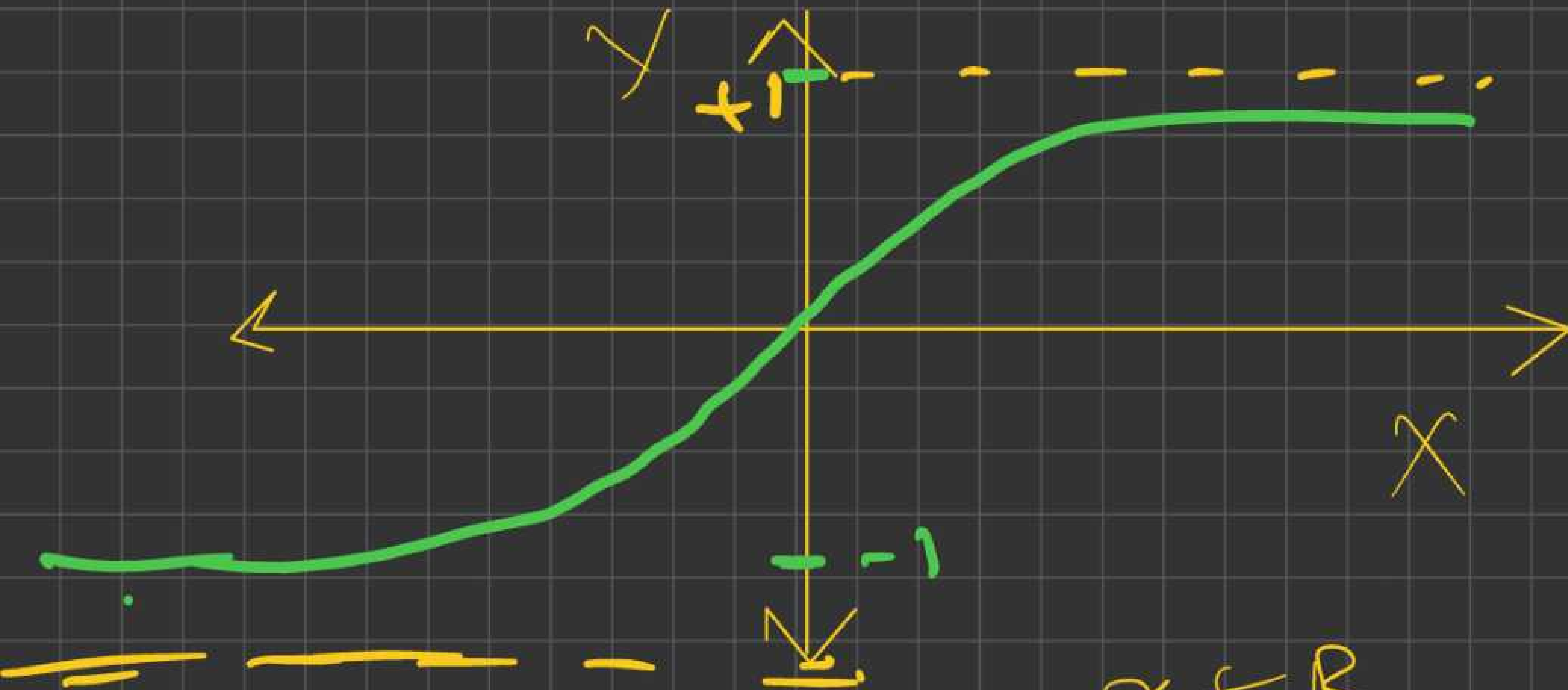
$$p = \frac{e^{z}}{1+e^{z}} \approx \frac{1}{1+e^{-z}}$$

$$\left(\frac{\cancel{e^{z}}}{(1+e^{z})} * \frac{e^{z}}{\cancel{e^{z}}} \right) \rightarrow \frac{1}{\frac{(1+e^{z})}{e^{z}}} \rightarrow \frac{1}{\left(\frac{1}{e^{z}} + 1\right)} \rightarrow \frac{1}{1+e^{-z}}$$

$$\boxed{\sigma(z) = \frac{1}{1+e^{-z}}}$$



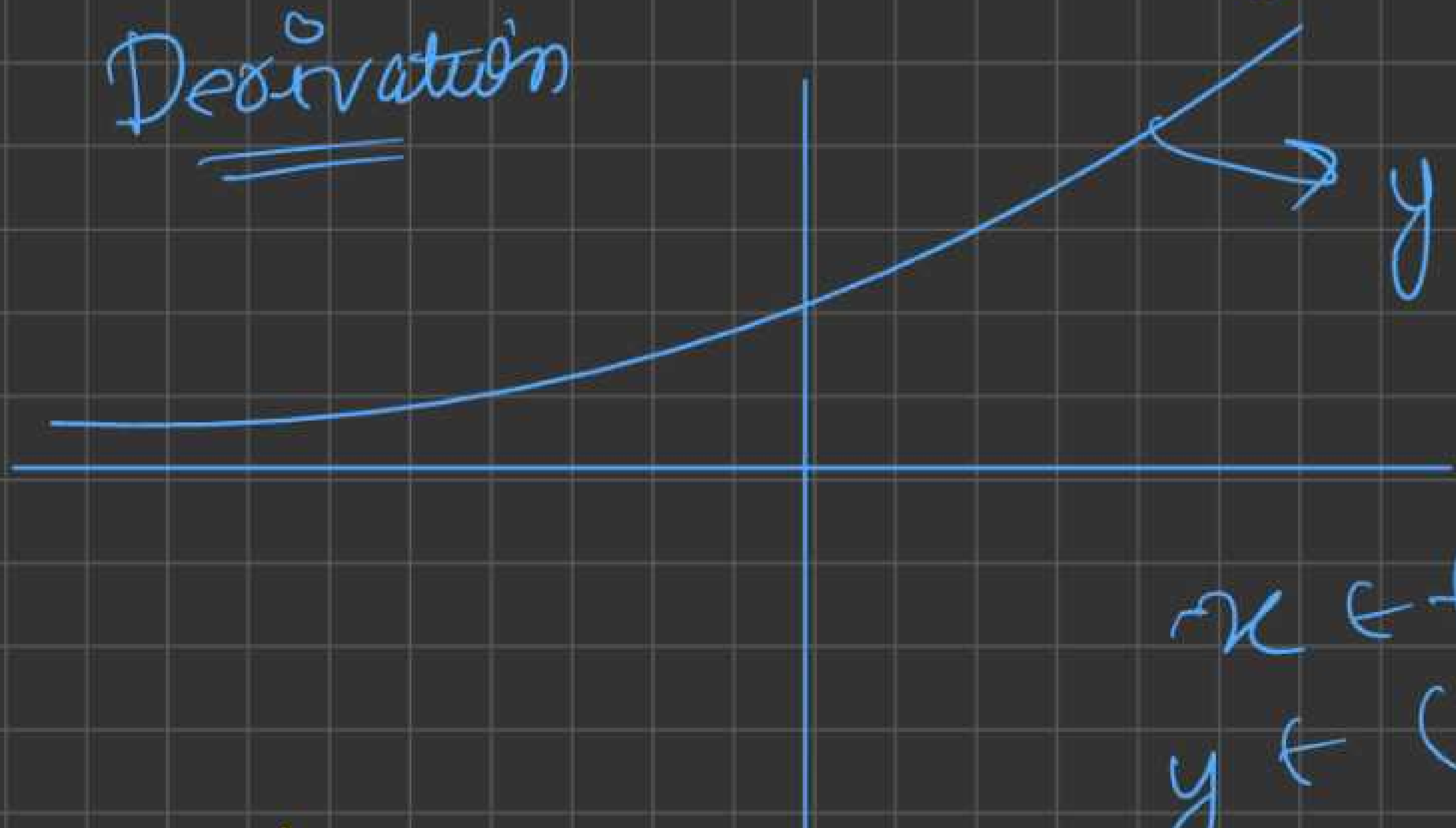
1) $\tanh = \left[\dfrac{e^{x} - e^{-x}}{e^{x} + e^{-x}} \right]$



$x \in R$

$y \in (-1, 1)$

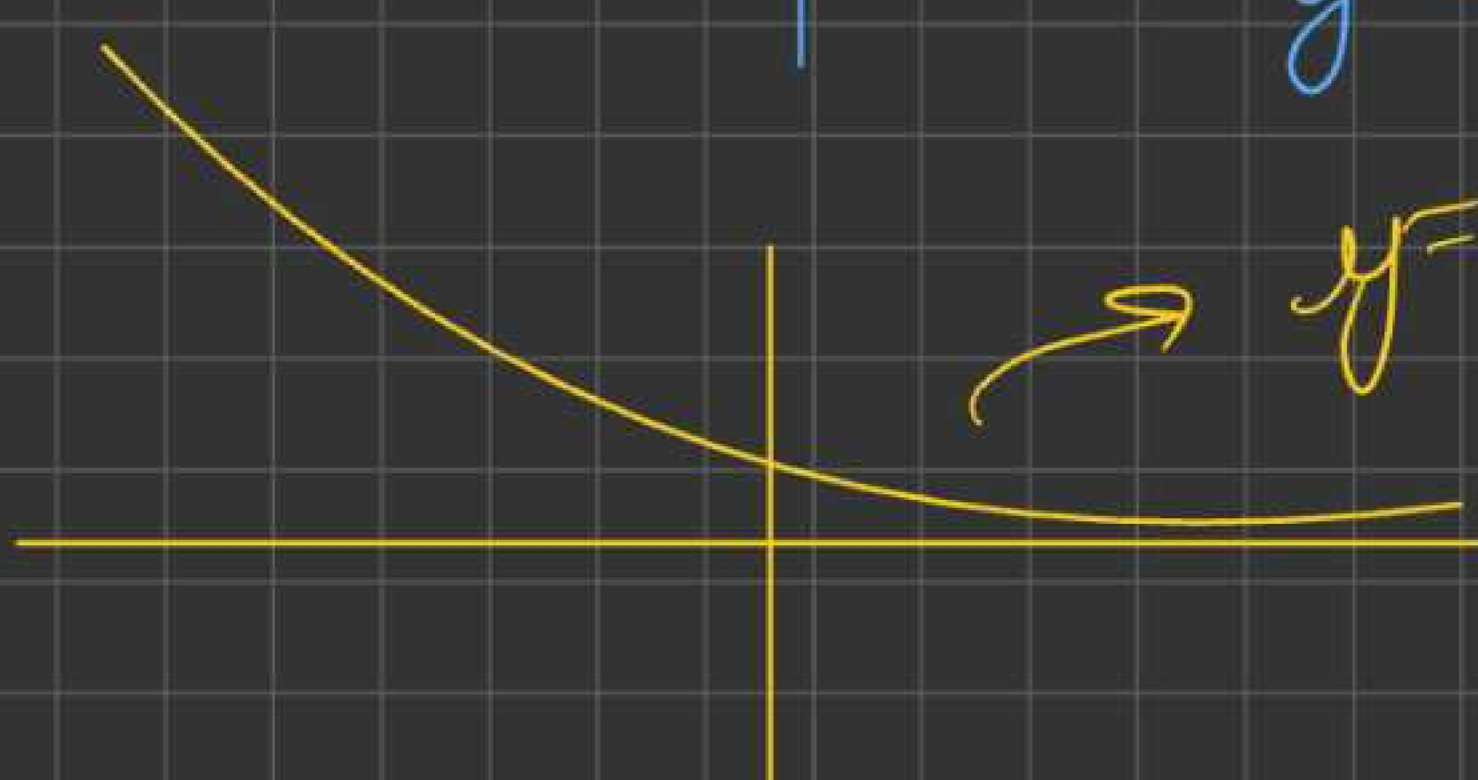
Derivation



$y = e^{x}$

$x \in R$

$y \in (0, \infty)$

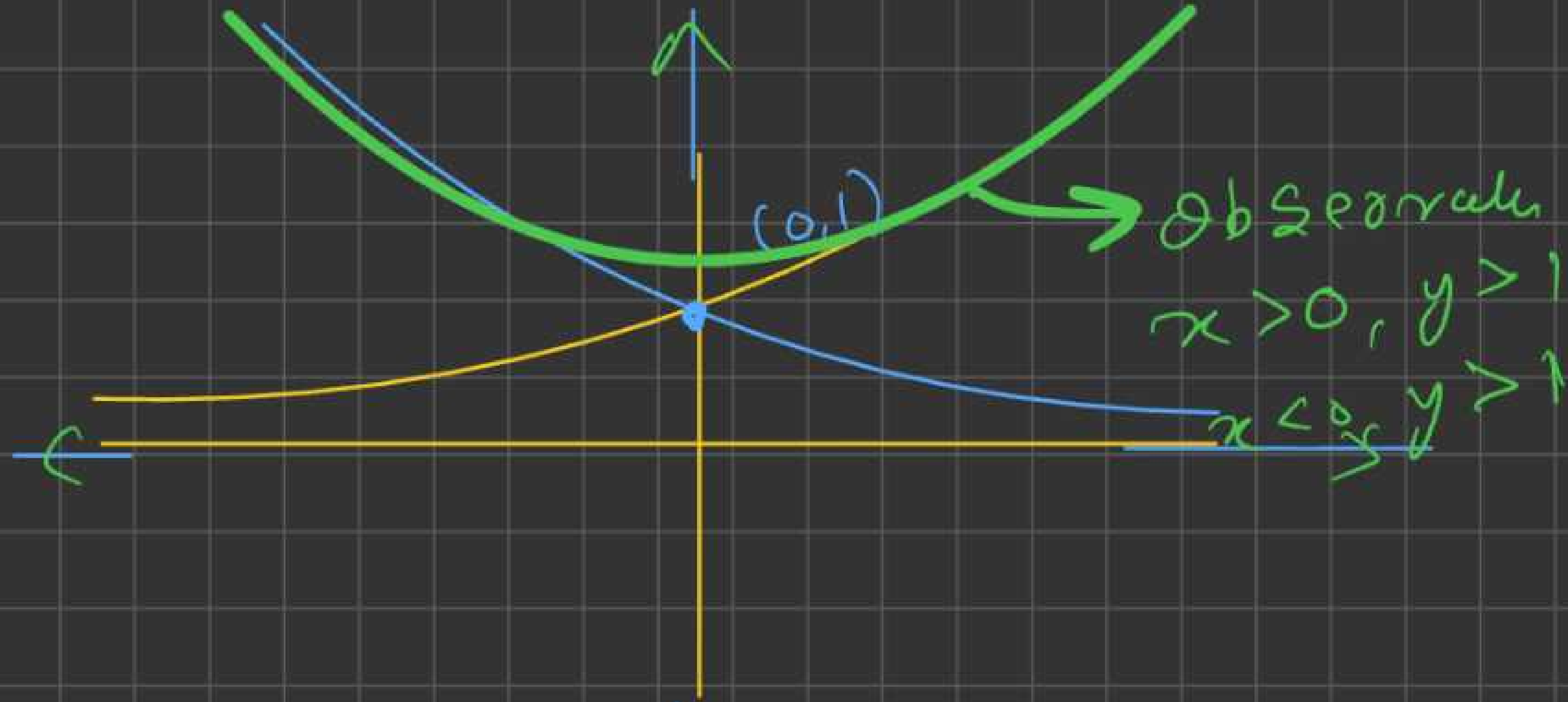


$y = e^{-x}$

$x \in R$

$y \in (0, \infty)$

A Rolm



$$f(x) = \frac{e^{x}+e^{-x}}{2}$$

from defⁿ & obs.

this new defⁿ is called / referred to as cosh

$$\cosh(x) = \frac{e^{x}+e^{-x}}{2} \rightarrow eq \rightarrow 1$$

Similarly

$$\sinh(x) = \frac{e^{x}-e^{-x}}{2} \rightarrow eq \rightarrow 2$$

$\cosh(x)$, $x>0$, $x<0$, $|x|=|-x|$, same output y will be there

$\sinh(x)$, $x>0$, $x<0$, $|x|=|-x|$, same output but diff. sign $|y|$ ✓

$$\tanh(x) = \frac{\sinh(x)}{\cosh(x)} = \frac{\left(\frac{e^{x} - e^{-x}}{\cancel{2}}\right)}{\left(\frac{e^{x} + e^{-x}}{\cancel{2}}\right)}$$

$$\boxed{\tanh(x) = \frac{e^{x} - e^{-x}}{e^{x} + e^{-x}}}$$ → Final formula

---

Q what is h in tanh, sinh, cosh?

A h → hyperbola.

$$\left(\frac{x}{a}\right)^2 - \left(\frac{y}{b}\right)^2 = 1$$

(Assuming $a = 1, b = 1$)

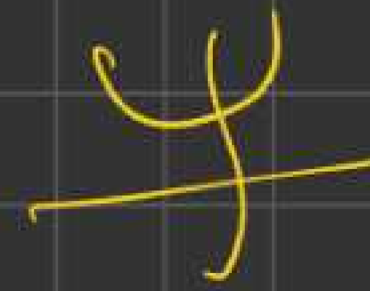
$$\boxed{x^2 - y^2 = 1}$$ → eq^n of hyperbola

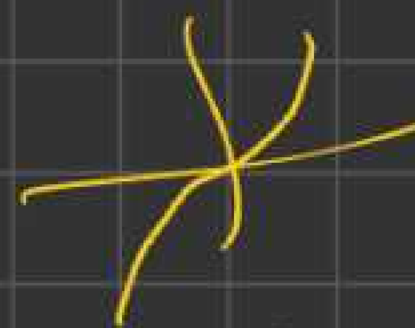
$\Rightarrow$

Made with Goodnotes

Q I want to find $\cosh^2 x - \sinh^2 x$

S

$$\cosh(x) = \frac{e^{x} + e^{-x}}{2}$$ 

$$\sinh(x) = \frac{e^{x} - e^{-x}}{2}$$ 

$$\cosh^2(x) = \left(\frac{e^{x} + e^{-x}}{2}\right)^2 = \boxed{\frac{e^{2x} + e^{-2x} + 2e^{x}e^{-x}}{4}}$$

$\rightarrow$ eq (1)

$$\sinh^2(x) = \left(\frac{e^{x} - e^{-x}}{2}\right)^2 = \boxed{\frac{e^{2x} + e^{-2x} - 2e^{x}e^{-x}}{4}}$$

$\rightarrow$ eq (2)

eq (1) − eq (2)

$$\cosh^2(x) - \sinh^2(x)$$

$$= \left(\frac{e^{2x} + e^{-2x} + 2e^{x}e^{-x}}{4}\right) - \left(\frac{e^{2x} + e^{-2x} - 2e^{x}e^{-x}}{4}\right)$$

$$= \frac{\cancel{e^{2x}} + \cancel{e^{-2x}} + 2e^{x}e^{-x} - \cancel{e^{2x}} - \cancel{e^{-2x}} + 2e^{x}e^{-x}}{4}$$

$$= \frac{2e^{x}e^{-x} + 2e^{x}e^{-x}}{4} = \frac{\cancel{4}e^{x}e^{-x}}{\cancel{4}}$$

$$= e^{x}e^{-x} = e^{0} = \underline{1}$$

conclusion

$$\left.\begin{array}{l}\cosh^2(x) - \sinh^2(x) = 1 \\ \text{we know that, } x^2 - y^2 = 1\end{array}\right]$$ Hyperbola Analogs

considering a circle,

$$(x-a)^2 + (y-b)^2 = r^2 \quad , \left(\begin{array}{l} r = 1 \\ a = 0, b = 0\end{array}\right)$$

$$x^2 + y^2 = 1$$

$$x = r\cos\theta, \quad y = r\sin\theta \quad (r = 1)$$

$$\cos^2\theta + \sin^2\theta = 1$$

---

circle	Hyperbola
$x^2 + y^2 = 1$	$x^2 - y^2 = 1$
$\cos^2\theta + \sin^2\theta = 1$	$\cosh^2(x) - \sinh^2(x) = 1$
$x = \cos\theta, \quad y = \sin\theta$	$x = \cosh(x)$ $y = \sinh(x)$

Note:- $\cos(x) \neq \cosh(x)$

$\sin(x) \neq \sinh(x)$

$\tan(x) \neq \tanh(x)$ → hyberbolic

circles

$$\boxed{\tanh(x) = \frac{e^{x} - e^{-x}}{e^{x} + e^{-x}}}$$

1) ReLU $\Rightarrow$ Rectified Linear Unit

$$\boxed{f(x) = ReLU = max(0, x)}$$

$\rightarrow$ Formula

$$f(x) = \begin{cases} x, & x > 0 \\ 0, & x \leq 0 \end{cases} \approx \begin{cases} x, & x \geq 0 \\ 0, & x < 0 \end{cases}$$

Both defn of ReLU are same i.e.

$$max(0, x) \approx \begin{cases} x, & x \geq 0 \\ 0, & x < 0 \end{cases}$$

why?

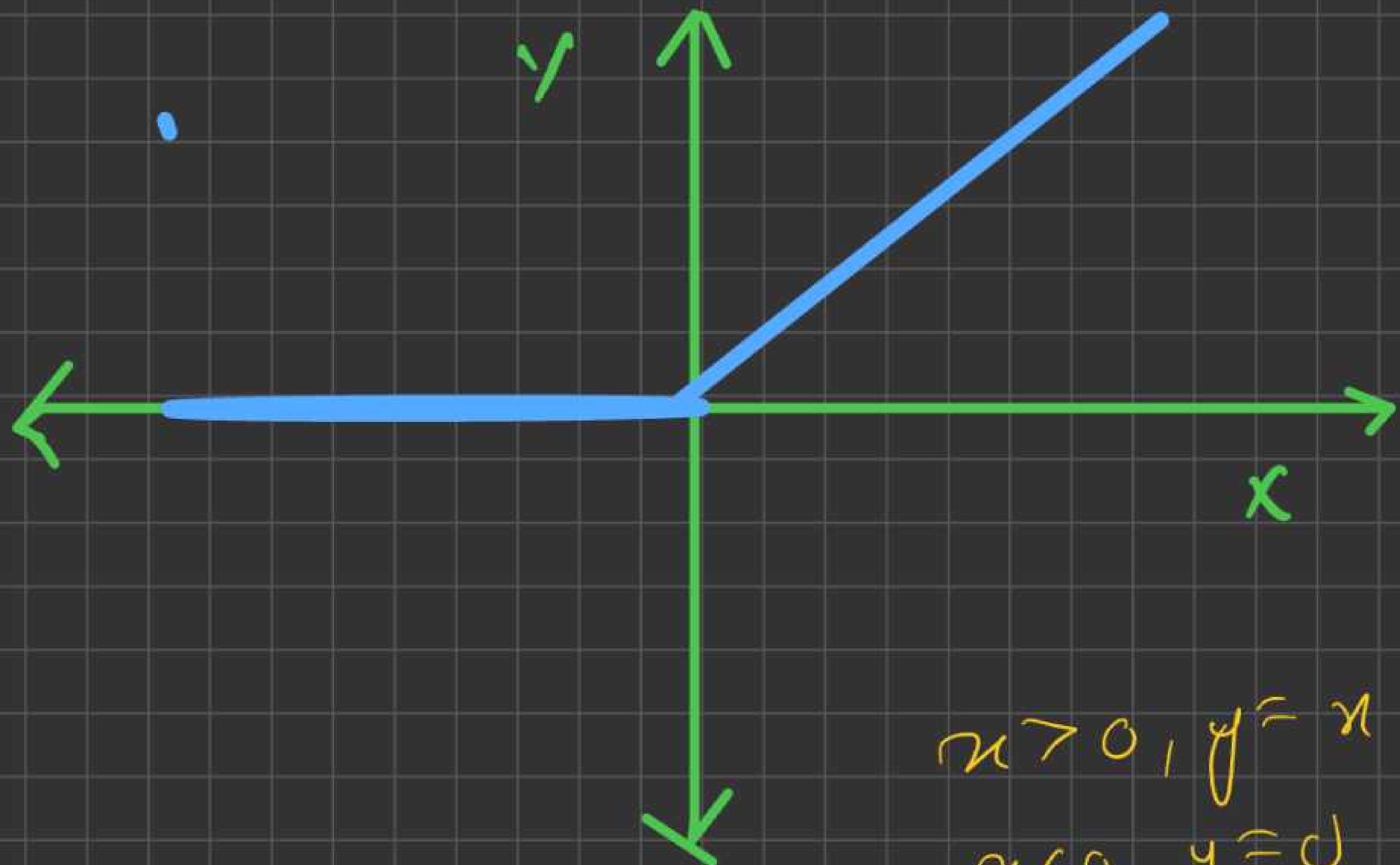
Because if $x > 0$, $max(0, x) = x$

if $x < 0$, $max(0, x) = 0$

for +ve input linear graph, $y = x$

-ve input, constant gf. $y = 0$

$\Rightarrow$



$x > 0, y = x$
$x < 0, y = 0$

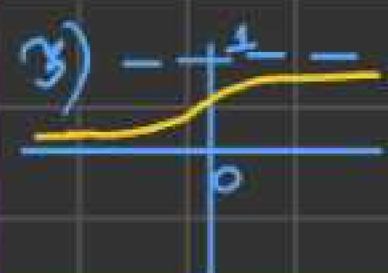
Q Have we introduced Non-linearity in this fun^n?

A- Non-linearity is there, as when
$x > 0$, we maintain linearity
$x < 0$, we do 0,

so Linear + Non-lin = Non-linear

Q Does squashing occur?
A- -ve signal ⇒ Block (some squash occur)
+ve signal → passed as it is.

ReLU is not much abt der^n but more about obs. & def^n.

Act^n Fu^n	Formula	Key char.	Pros	Cons	Use case
Sigmoid	$\sigma(z) = \frac{1}{1+e^{-z}}$	1) output b/w $\Rightarrow (0,1)$ 2) S-shape curve 3) 	1) Smooth curve (easily interpretable) 2) Outputs are represented as probabilities	1) vanishing gradients (slow learning) 2) Not zero-centred.	1) classification problems 2) older models
tanh	$\tanh(z) = \frac{e^{z}-e^{-z}}{e^{z}+e^{-z}}$	1) output -1 & 1.. 2) S-shape curve 3) 	1) Smooth curve. (easily interpretable) 2) fun^n is zero centric	1) Prone to vanishing gradients	1) Hidden layers in RNN it is used
ReLU (Rectified linear unit)	$f(z) = \max(0,z)$	1) if $z>0$, $y=z$ (linear) 2) if $z<0$ (neuron off)	1) Fast computation 2) No-vanishing gradient for +ve vals	1) Dying Neuron problem. 2) Not smooth	1) most modern networks hidden layers have this Act^n fun^n (like transformer)

implementing activatⁿ funⁿ in code

1) sigmoid = $\frac{1}{1+e^{-z}}$ = $\frac{1}{1+\text{math.exp}(-z)}$

case I :-

z = value (2.0)

$$\boxed{= \frac{1}{1+(\text{value}(2.73)**(-z.\text{data}))}}$$

→ 1st design choice

implementing from prev ops

case II z = value (2.0)

$$\boxed{s = \frac{1}{1+\text{math.exp}(-z.\text{data})}}$$

Value (s, (z,), 'sigmoid') → second design choice

we are going with 2nd design choice
as it seems to better choice bcz on backward
we had to do less # of ops as complex funⁿ

Made with Goodnotes

derivatives can be simple

if we go with design choice 1, then we will end up doing more # of ops, in a Value object for *, +, - etc.

$$\tanh = \frac{e^{-z}\left(e^{z} - e^{-z}\right)}{e^{-z} * \left(e^{z} + e^{-z}\right)}$$ (dividing entire eqn by e^{-z} & multiply by e^{-z})

$$= \boxed{\frac{e^{2z} - 1}{e^{2z} + 1}}$$

Ist choice

I) ((Value(2.71) ** (2 * self.data) - 1) / (Value(2.71) ** (2 * self.data) + 1))

→ Design I → Bad choice

as Back prop will be having so many int. go to calculate

$$z = \boxed{\frac{\text{math.exp}(2 * \text{self.data}) - 1}{\text{math.exp}(2 * \text{self.data}) + 1}}$$

→ good design choice

Value(z, (self,), 'tan')

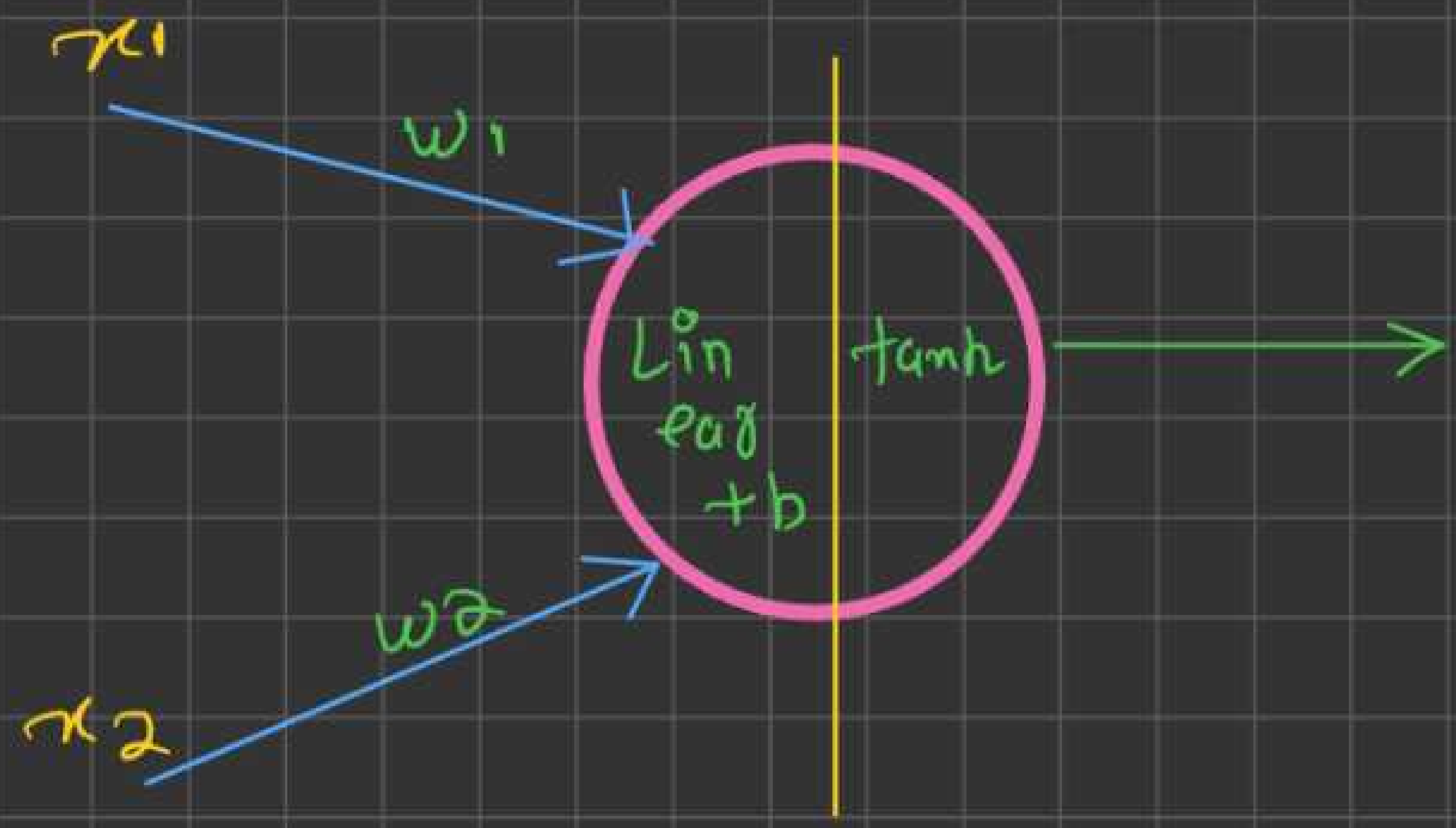
1 on calculation of derivative of

$$\frac{d}{dz}\left(\frac{e^{2z}-1}{e^{2z}+1}\right) = (1 - z^2)$$

→ so only 2 opn, (-), (*+)

So, designⁿ choice 2 is better as total grad. calculation is less ⇒

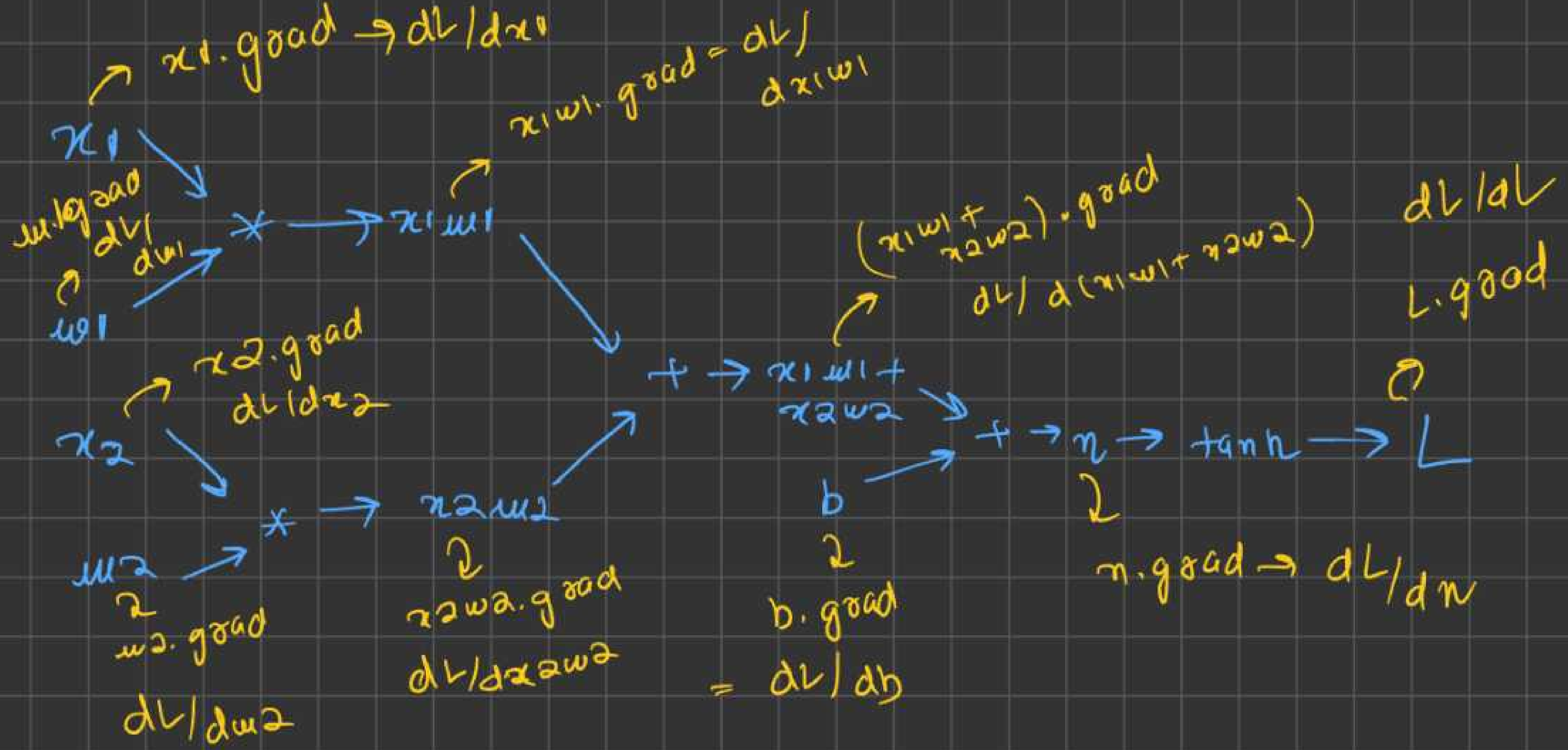
Forward & Backward Pass manual calculation example with one neuron & Act^n fun^n & multiple input & weight.



$x_1, x_2, w_1, w_2, b,$ tanh (given)

$$\boxed{\begin{aligned} x1w1 &= x1 * w1 \\ x2w2 &= x2 * w2 \\ x1w1x2w2 &= x1w1 + x2w2 \\ n &= x1w1x2w2 + b \\ L &= n.\text{tanh}() \end{aligned}}$$

→ forward pass



A) L.grad = ?   dL/dL = ?

$$\text{slope} = \frac{f(x+h) - f(x)}{h} = \frac{\cancel{L} + h - \cancel{L}}{h} = 1$$

$\boxed{\text{slope} = 1}$ → Analytically

B) $\frac{dL}{dL} \Rightarrow \frac{dL}{dL} = 1$, $(1x^1 = 1x^0 = 1)$
$dx/(dx = 1)$

$\boxed{\text{slope} = 1}$ → calculus

B) $x.grad = \frac{\partial L}{\partial x}$ $L = \tanh(x)$

$$\frac{\partial L}{\partial x} = \text{slope} = \frac{f(x+h) - f(x)}{h}$$

$$= \boxed{\frac{\tanh(x+h) - \tanh(x)}{h}}$$

→ skipping & moving to cal.

we know that,

$$\tanh(x) = \frac{e^{x} - e^{-x}}{e^{x} + e^{-x}}$$

$$\sinh(x) = (e^{x} - e^{-x}) / 2$$

$$\cosh(x) = (e^{x} + e^{-x}) / 2$$

$$\frac{\partial \tanh(x)}{\partial x} = \frac{\partial}{\partial x}\left(\frac{e^{x} - e^{-x}}{e^{x} + e^{-x}}\right)$$

Made with Goodnotes

B) Calculate, $\frac{\partial L}{\partial x} = ?$, x.grad, $L = \tanh(x)$

$$\frac{\partial}{\partial x}\tanh(x) = \frac{\partial}{\partial x}\left(\frac{e^{x} - e^{-x}}{e^{x} + e^{-x}}\right)$$

(we know that,

$$\left(d(xy) = x\,dy + y\,dx\right)$$

$$\left(d(x/y) = \frac{y\,dx - x\,dy}{y^2}\right)$$

$$\frac{\partial}{\partial x}\tanh(x) = \frac{(e^{x}+e^{-x})\,\partial(e^{x}-e^{-x}) - (e^{x}-e^{-x}) * \partial(e^{x}+e^{-x})}{(e^{x}+e^{-x})^2}$$

(we know that, $d(e^{x}) = e^{x}$

$$\left(e^{-x} = -e^{-x}\right)$$

So, $\partial(e^{x} - e^{-x}) = (e^{x} + e^{-x})$

$$\partial(e^{x} + e^{-x}) = (e^{x} - e^{-x})$$

$$\frac{\partial}{\partial x}\left(\tanh(x)\right) = \frac{(e^{x}+e^{-x})(e^{x}+e^{-x}) - (e^{x}-e^{-x})(e^{x}-e^{-x})}{(e^{x}+e^{-x})^2}$$

$$= \frac{(e^{x}+e^{-x})^2 - (e^{x}-e^{-x})^2}{(e^{x}+e^{-x})^2}$$

$$= \cancel{\frac{(e^{x}+e^{-x})^2}{(e^{x}+e^{-x})^2}} - \left(\frac{(e^{x}-e^{-x})}{e^{x}+e^{-x}}\right)^2$$

$$= 1 - \left(\frac{e^{x}-e^{-x}}{e^{x}+e^{-x}}\right)^2$$

$$\frac{\partial \tanh(x)}{\partial x} = 1 - (\tanh(x))^2$$

calculus

$$\boxed{\frac{\partial L}{\partial x} = x.grad = (1-\tanh(x))^2}$$

$$= (1 - L^2) \quad (L = \tanh(x))$$

$$= (1 - (0.7071)^2)$$

$$= 0.5$$

back to slope

$\frac{\partial L}{\partial x} = ?$, $L = \tanh(x)$

$h \to 0$

$\to h \approx 0$

$$\text{slope} = \frac{f(x+h) - f(x)}{h} = \frac{\tanh(x+h) - \tanh(x)}{h}$$

(we know that,

$$\tanh(A+B) = \frac{\tanh(A) + \tan(B)}{1 + \tanh(A)\tanh(B)})$$

$$= \lim_{h \to 0} \frac{\frac{\tanh(x) + \tanh(h)}{1 + \tanh(x)\tanh(h)} - \tanh(x)}{h}$$

$$= \left(\frac{\frac{L + \tanh(h)}{1 + L\tanh(h)} - L}{h} \right)$$

Made with Goodnotes

$$= \frac{\dfrac{L + \tanh(h)}{1 + L\tanh(h)} - L}{h}$$

$$= \frac{L + \tanh(h) - L\left(1 + L\tanh(h)\right)}{h\left(1 + L\tanh(h)\right)}$$

$$= \frac{\cancel{L} + \tanh(h) - \cancel{L} - L^2\tanh(h)}{h\left(1 + L\tanh(h)\right)}$$

$$= \frac{\tanh(h)\left(1 - L^2\right)}{h\left(1 + L\tanh(h)\right)}$$

$$= \lim_{h \to 0} \left(\frac{\tanh(h)}{h}\right)^{1} \left(1 - L^2\right) \left(\frac{1}{1 + L\tanh(h)}\right)^{1}$$

(we know that, $\lim_{x \to 0} \dfrac{\tan(x)}{x} = 1$)

$$\left(\frac{1}{1 + L\tanh(h)} = 1 \right) \quad \tanh(h) \to 0, \; h \to 0$$

$$\frac{\partial L}{\partial n} = \boxed{(1-L^2)} = 1-(0.7)^2 = (0.5)$$

Analytically

b.grad = ? $\qquad \frac{\partial L}{\partial b} = ?$, $n = (x_1 w_1 + x_2 w_2) + b$

chain rule

$$\frac{dz}{dx} = \left(\frac{dy}{dx}\right) * \left(\frac{dz}{dy}\right)$$

$$\frac{\partial L}{\partial b} = \left(\frac{\partial n}{\partial b}\right) * \left(\frac{\partial L}{\partial n}\right)$$

b.grad = $\left(\frac{\partial n}{\partial b}\right)$ * n.grad

$\frac{\partial n}{\partial b} = ?$. A) slope = ?

$$\frac{f(b+h) - f(b)}{h} = \frac{\cancel{(x_1 w_1 + x_2 w_2)} + (\cancel{b}+h) - (\cancel{(x_1 w_1 + x_2 w_2)} + \cancel{b})}{h}$$

$= 1$

$$\boxed{\frac{\partial n}{\partial b} = \text{local der.} = 1}$$

b.grad = n.grad ☆

Made with Goodnotes

on, observation, when a add^n happens the local derivative turns out to be equal to 1.

(+) → gradient remain Same

b.grad = n.grad.

if b.grad = n.grad,

so, from above obs, we can say

$$\boxed{(x_1w_1 + x_2w_2).grad = b.grad = \underline{n}.grad = 0.5}$$

$x_1w_1 + x_2w_2 = x_1w_1 + x_2w_2$

$x_1w_1.grad = ?$, $x_2w_2.grad = ?$

From above property, as add^n happens,

so, $\boxed{(x_1w_1 + x_2w_2).grad = x_1w_1.grad = x_2w_2.grad}$
$= 0.5$

Made with Goodnotes

$$x2w2 = x2 * w2$$

$$\frac{\partial L}{\partial x2} = ?$$

$$\frac{\partial L}{\partial x2} = \frac{\partial (x2w2)}{\partial x2} * \frac{\partial L}{\partial (x2w2)}$$

$$x2.grad = \left(\frac{\partial (x_2 w_2)}{\partial x_2}\right) * x2w2.grad$$

$$\frac{\partial (x_2 w_2)}{\partial x2} = ?$$

$$slope = \frac{f(x+h) - f(x)}{h}$$

$$= \frac{(x2+h)w2 - (x2w2)}{h}$$

$$= (\cancel{x2w2} + hw2 - \cancel{x2w2}) / \cancel{h}$$

$$\boxed{= w2} \rightarrow \text{local der.}$$

Analytically

$$\boxed{x2.grad = w2 * (x2w2).grad}$$

Made with Goodnotes

As per obs. if '*' comes, then multiply with the opp. term to which grad. has to be calc. with the global derivative, as opp. term/data is the local der.

$$c = a * b$$

$$a.grad = b * c.grad$$

so,

$$x2.grad = (w2) * (x2w2).grad$$

$$w2.grad = (x2) * (x2w2).grad$$

$$x2.grad = 1 * 0.5 = 0.5$$

$$w2.grad = 0 * 0.5 = \cancel{0.5}\ 0$$

$$x1.grad = (w1) * (x1w1).grad$$

$$w1.grad = (x1) * (x1w1).grad$$

$$x1.grad = -3 * 0.5 = -1.5$$

$$w1.grad = 2 * 0.5 = 1$$

A) Pow

$$y = x^n$$

$$\frac{dy}{dx} = \frac{d}{dx}(x^n) = \boxed{n * x^{(n-1)}}$$

$$\frac{\partial L}{\partial x} = \frac{\partial y}{\partial y} * \frac{\partial L}{\partial y}$$

$$\boxed{x.grad = \underline{(n * x^{(n-1)})} * \underline{y.grad}}$$

B) ReLU

$$y = max(0, x)$$

$$\frac{dy}{dx} = ?\ 0,\ 1 \qquad y = \begin{cases} x, & x > 0 \\ 0, & x < 0 \end{cases}$$

$$\boxed{\begin{aligned} \frac{dy}{dx},\ x > 0 &= \frac{dx}{dx} = \underline{1} \\ \frac{dy}{dx},\ x \leq 0,\ & \frac{d(x=k)}{dx} = 0 \end{aligned}}$$

Made with Goodnotes

$$\frac{\partial L}{\partial x} = \frac{\partial y}{\partial x} * \frac{\partial L}{\partial y}$$

$$x.grad = \frac{\partial y}{\partial x} * y.grad$$

$x.grad = y.grad \quad (x > 0)$
$(\partial y / \partial x = 1)$

$x.grad = 0 \quad (x \leq 0, \ \partial y / \partial x = 0)$

if $x > 0$:
  $x.grad = y.grad$
else:
  $x.grad = 0$

---

* $y = \dfrac{1}{1+e^{-x}}$ (sigmoid)

$$\frac{dy}{dx} = \frac{d}{dx}\left(\frac{1}{1+e^{-x}}\right)$$

(we know that, $d(x/y) = \dfrac{y\,dx - x\,dy}{y^2}$)

$$\frac{d}{dx}\left(\frac{1}{1+e^{-x}}\right) = \frac{(1+e^{-x})\cancelto{0}{d(1)} - (1)\,d(1+e^{-x})}{(1+e^{-x})^2}$$

$$= \frac{-d(1+e^{-x})}{(1+e^{-x})^2} = \frac{-(d(1)+d(e^{-x}))}{(1+e^{-x})^2}$$

$$= \frac{+e^{-x}}{(1+e^{-x})^2} = e^{-x} * \left(\frac{1}{1+e^{-x}}\right)^2$$

$$= e^{-x} * y^2$$

$$= (1+e^{-x}-1) * y^2$$

$$\rightarrow \left(y = \frac{1}{1+e^{-x}},\quad (1+e^{-x}) = \frac{1}{y} \right)$$

$$= \left(\frac{1}{y} - 1\right) * y^2 = \frac{(1-y)}{\cancel{y}} * y^{\cancel{2}}$$

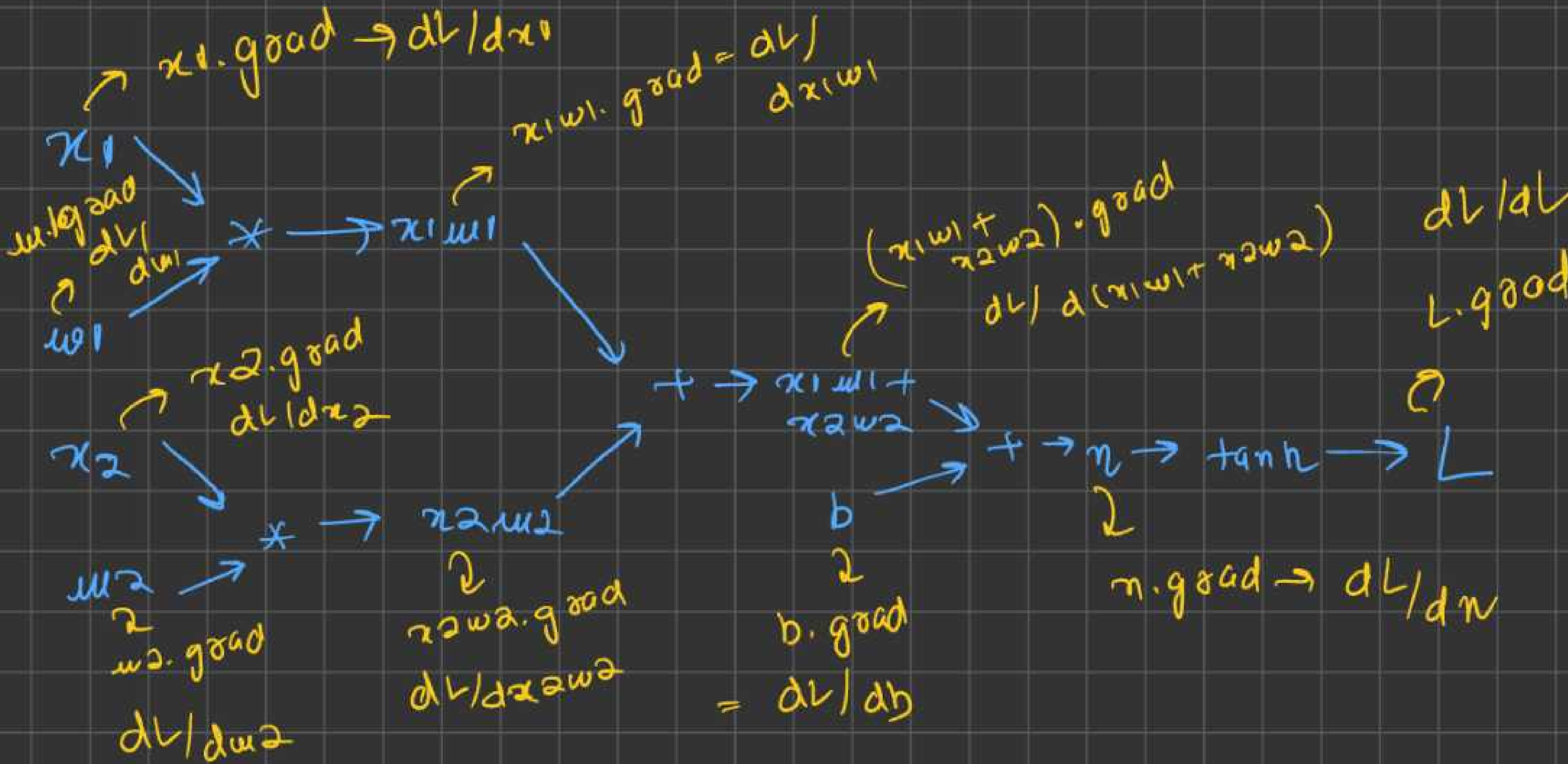
$$= \boxed{(1-y) * y}$$

$$\frac{\partial L}{\partial x} = \frac{\partial y}{\partial x} * \frac{\partial L}{\partial y}$$

$$x.grad = \frac{\partial y}{\partial x} * y.grad$$

$$\boxed{x.grad = y * (1-y) * y.grad}$$

Topological sort for Neural Network



Important:

1) Neural Networks can never have cycles in these computational graph

2) So, these graphs are always DAG

DAG → Directed Acyclic Graph

L

n

$x_1w_1 + x_2w_2$

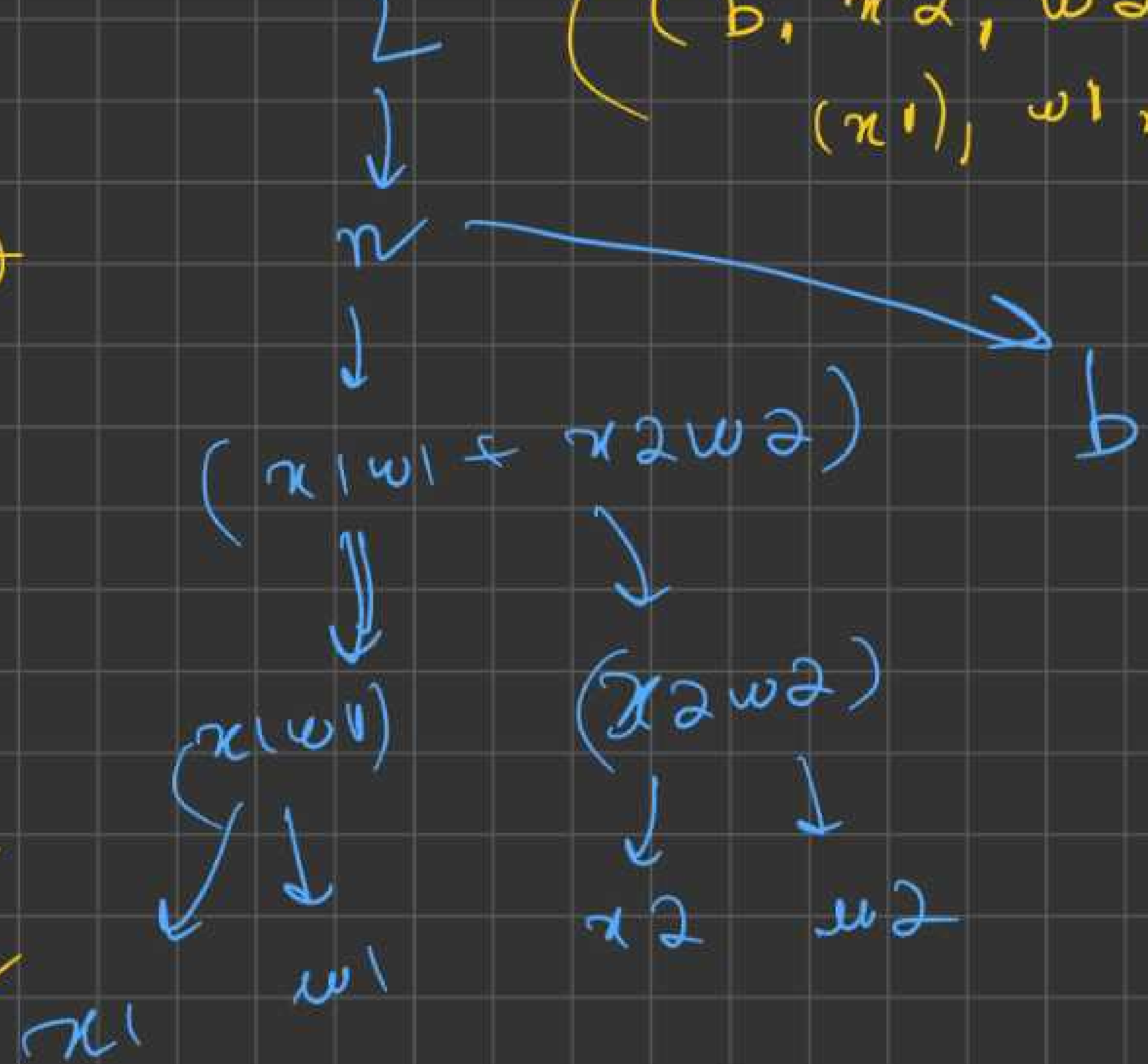
x_1w_1

$x_1 \to w_1$
$w_1 \to x_1$

x_2w_2

$x_2 \to w_2$
$w_2 \to x_2$

b

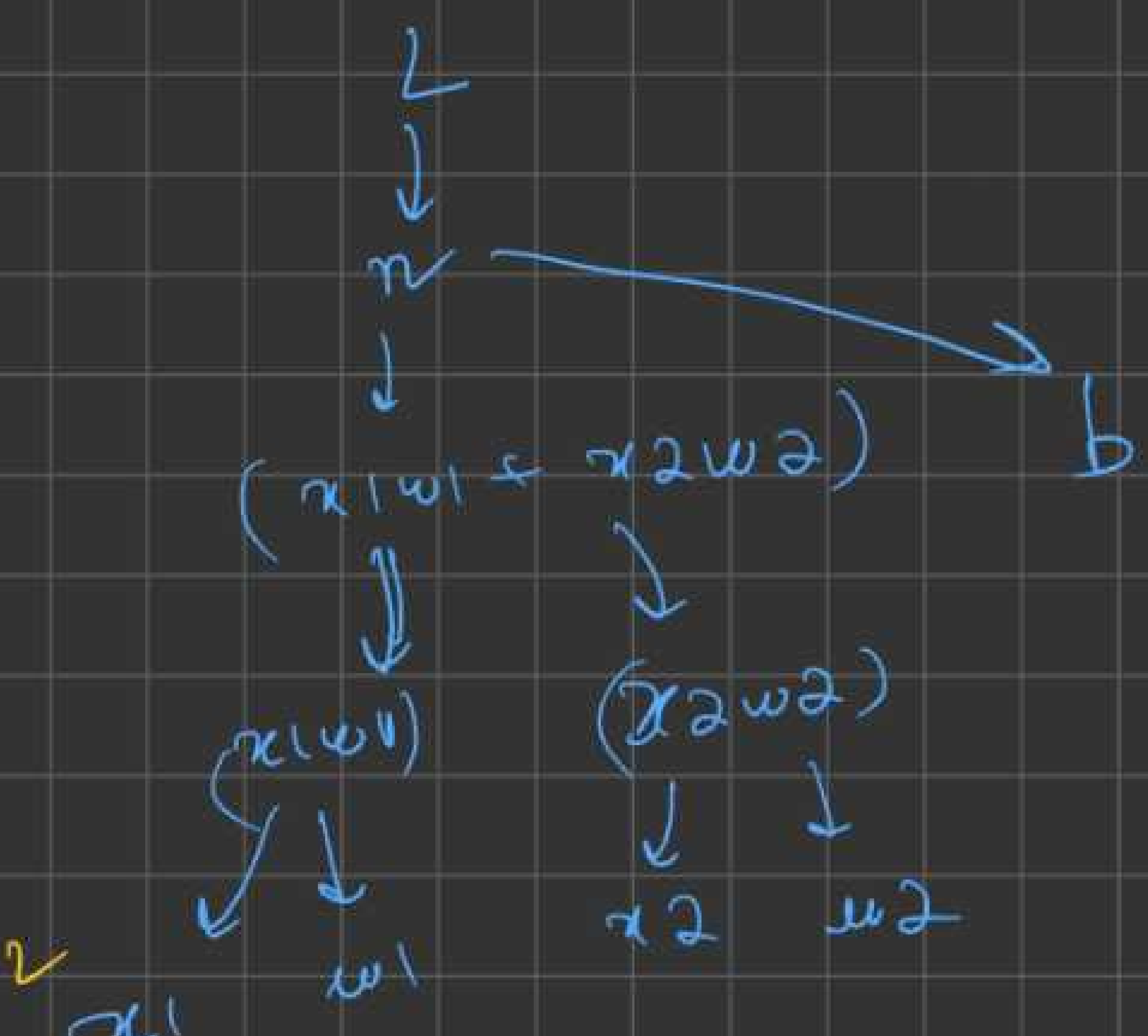


((b, x_2, w_2, (x_2w_2),
(x_1), w_1, (x_1w_1),
$(x_1w_1 + x_2w_2)$,
n, L)

```
topo = []
visited = set()
def build_topo(v):
    if v not in visited:
        visited.add(v)
        for child in v._prev:
            build_topo(child)
        topo.append(v)
```

Made with Goodnotes

```
topo = []
visited = set()
def build_topo(v):
    if v not in visited:
        visited.add(v)
        for child in v._prev:
            build_topo(child)
        topo.append(v)
```



Dry run

topo = []

visited = set()

~~build_topo~~(L)

visited = (L)

L._prev = [n]

~~build_topo~~(n)

visited = (L, n)

n.prev = [x1w1 + x2w2, b]

build_topo(x1w1 + x2w2)

visited = (L, n, x1w1 + x2w2)

x1w1 + x2w.prev = [x1w1, x2w2]

~~build topo~~ (x1w1).

visited = (x, L, x1w1 + x2w2, x1w1)

x1w1.prev = [x1, w1]

build topo (x1)

Made with Goodnotes

visited. ~~ad~~ (L, n, ($x_1w_1 + x_2w_2$, x_1w_1, x_1)

topo. = [x_1]

~~build topo~~ (w_1)

vis = (L, n, ($x_1w_1 + x_2w_2$), x_1w_1, x_1, w_1)

topo = [x_1, w_1]

---

topo. append = [x_1, w_1, x_1w_1]

~~buil topo~~ (x_2w_2)

vis = (L, n, ($x_1w_1 - x_2w_2$), x_1, w_1, x_1w_1 x_2w_2)

x_2w_2. prev = [x_2, w_2]

~~build topo~~ (x_2)

vis =

(L, n, ($x_1w_1 - x_2w_2$), x_1, w_1

x_1w_1 x_2w_2)

, x_2.

topo = [x1, w1, x1w1, x2]

~~build topo (x2)~~

~~x2~~ vis = (x1, w1, x1w1 + ~~x2~~ x2w2,
L, n, x2, x2w2)

topo = [x1, w1, x1w1, x2, w2]

---

topo = [x1, w1, x1w1, x2, w2, x2w2]

---

topo = [x1, w1, x1w1, x2, w2, x2w2,
x1w1 + x2w2]

~~build_topo~~ (b)

vis = (x1, w1, x1w1, x2, w2, x2w2,
(x1w1 + x2w2), L, n, b)

topo = [x1, w1, x1w1, x2, w2,
x2w2, x1w1 + x2w2, b]

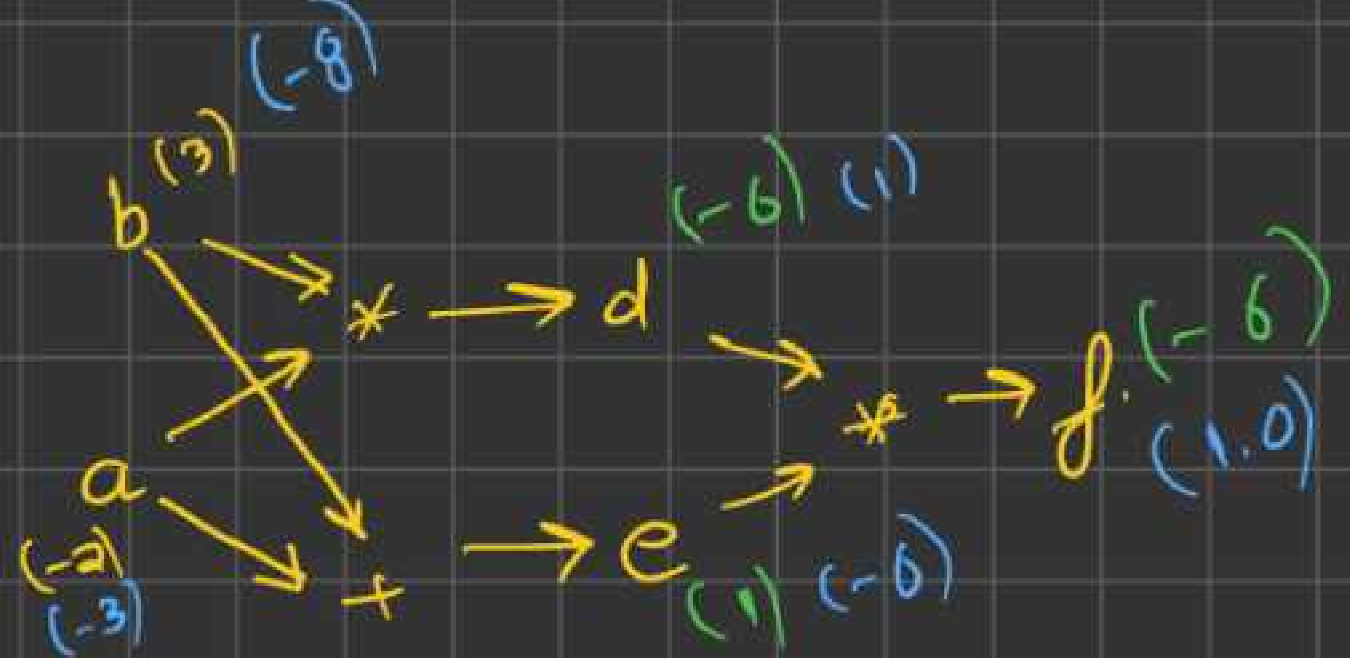
Made with Goodnotes

$$\boxed{\text{topo} = [x1, w1, x1w1, x2, w2, x2w2, x1w1 + x2w2, b, n, L]}$$

Final topo sort

Bug Fix (observation)
→ gradient overwriting

(gradient overwriting problem)



$d = a * b = -2 * 3 = -6$

$e = a + b = -2 + 3 = 1$

$f = d * e = 1 * (-6) = -6$

(So, forward Pass seems to be matching with the code)

$f.grad = 1.0$

$e.grad = \frac{df}{de} * f.grad = (-6) * 1 = -6$

$d.grad = \frac{df}{dd} * f.grad = 1.0$

$a.grad = \frac{dd}{da} * d.grad + \frac{de}{da} * e.grad$

$= 3 + (-6) = -3$

$b.grad = \frac{dd}{db} * d.grad + \frac{de}{da} * e.grad$

$= (-2) * 1 + (-6) = -8$

Made with Goodnotes

Intro. to Diff.
Deel. Library

Most famous dL libr. is Pytorch.
Develop by Meta
in FAIR lab, one of the co-founder
of this lab is Yann LeCunn. → AI researcher.
↓
indian (Somith
chontala)
→ creator of
Pytorch

Pytorch has tensors
has modules / sub-module

★ in python, we have 64 bit numbers

but torch.tensor is 32 bit number by default

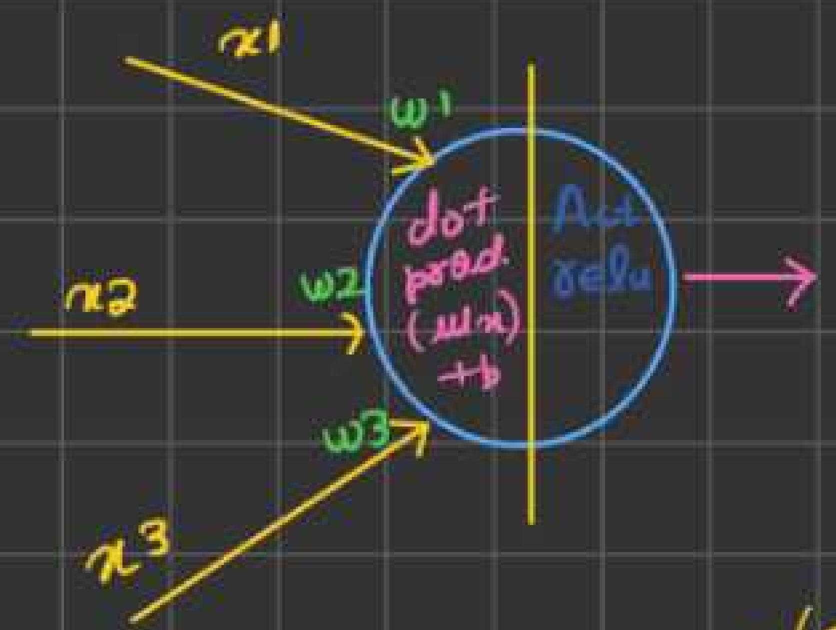
to convert to 64 bit use double()

Implementing Neural Network class (nn)

Neuron

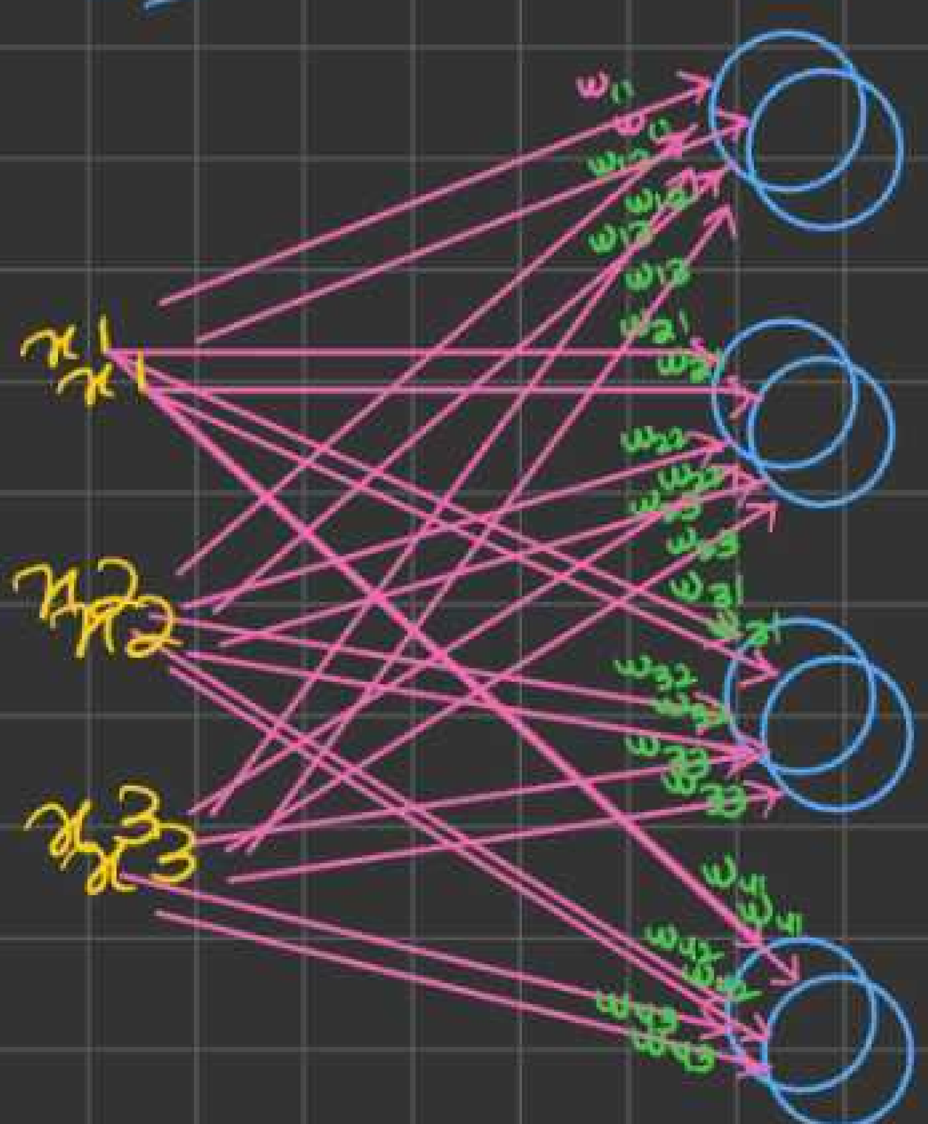
$w.x = w_1x_1 + w_2x_2 + w_3x_3 + b$

(b= bias)



Layer

$$\text{parameter} = (\underset{3\,(x_1\,x_2\,x_3)}{nin} \times nout) + nout$$



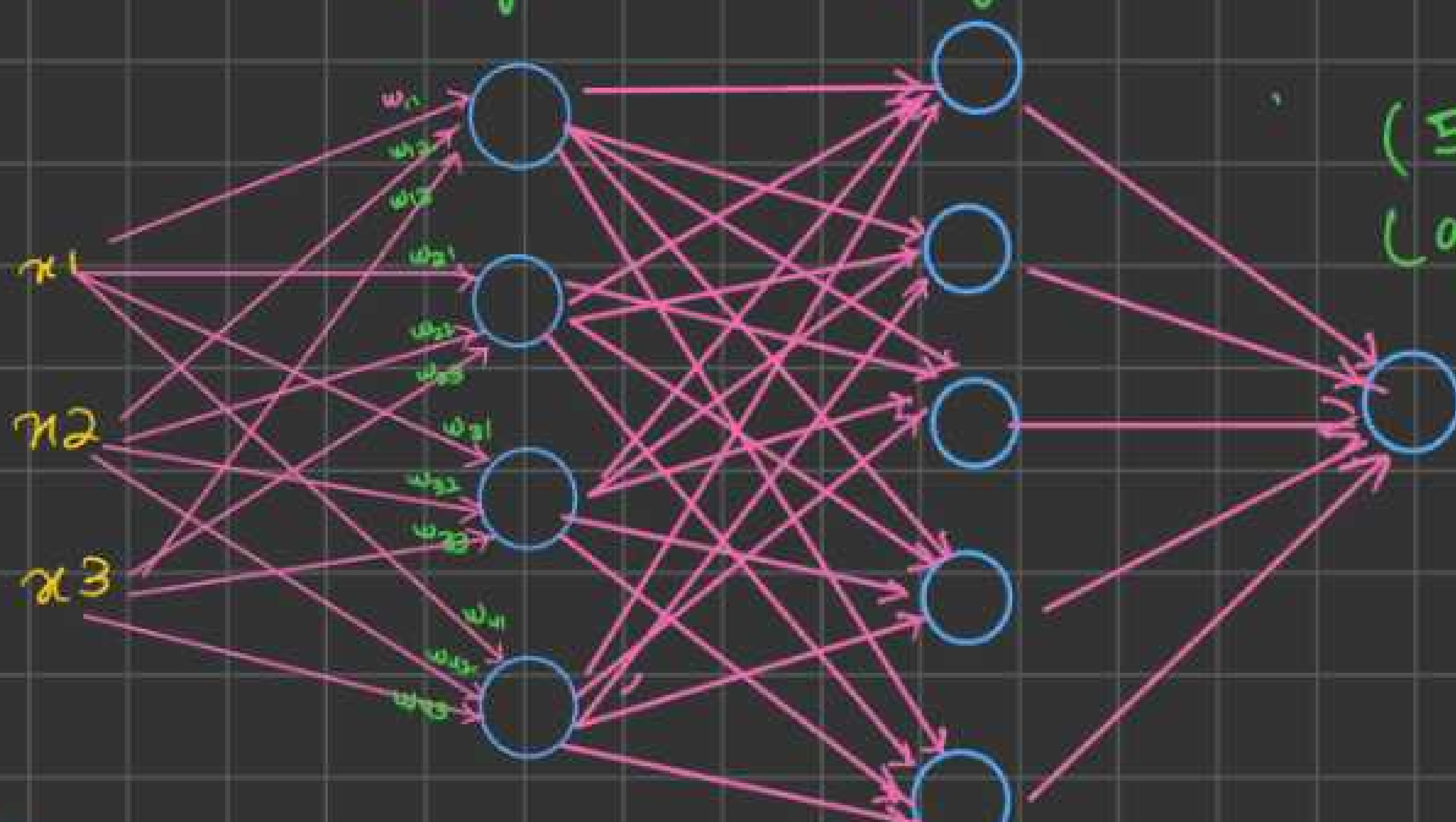
layer I = (layer (I-1)[1] , —)

MLP

(3,4) Layer I

(4,5) Layer II

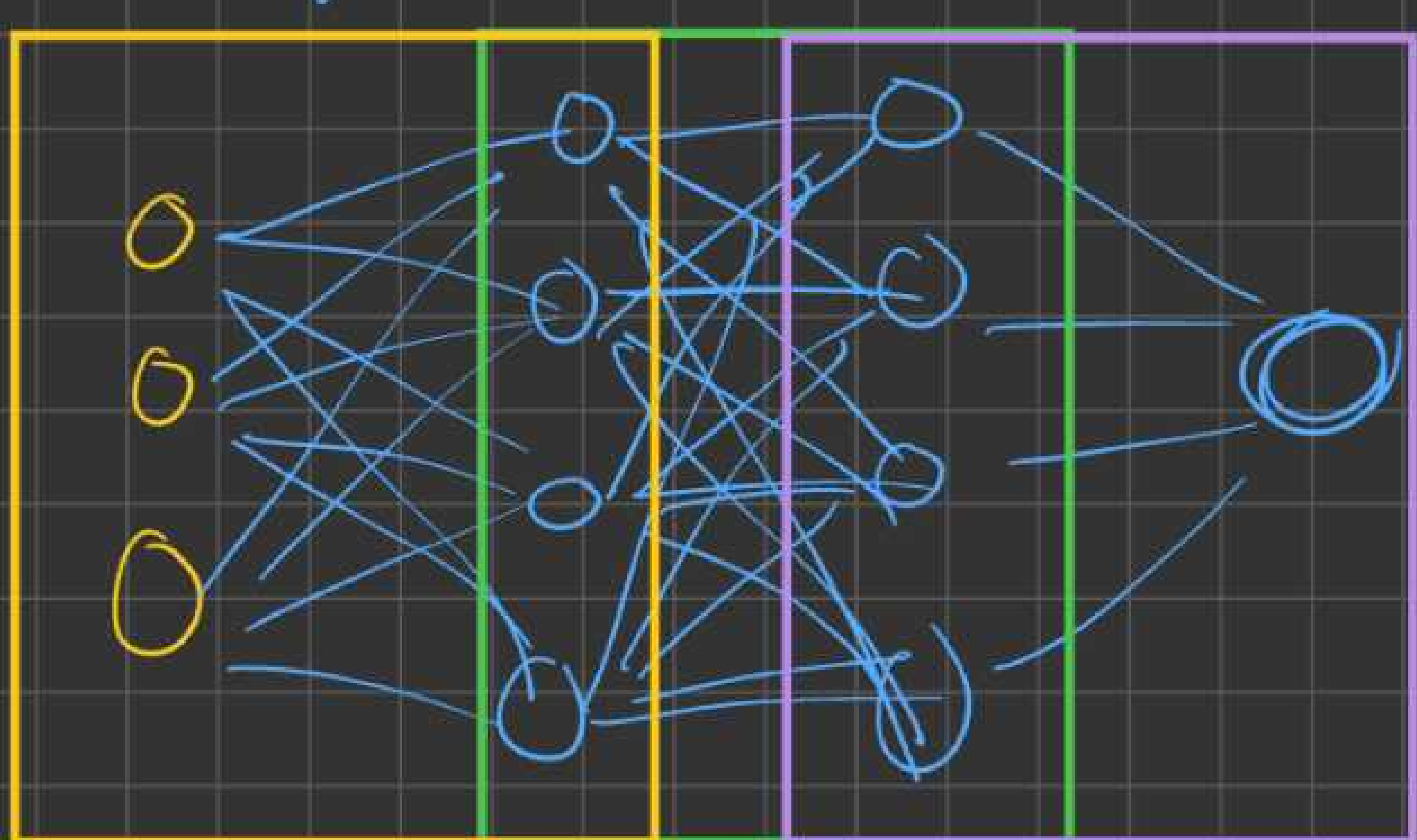
(5,1) Layer III



Training a Neural Network

First (regression)

My network size



(3, 4) (4, 4) (4, 1)

(A) After forward pass, we need to calculate loss in the predictions

ys = [A, B, C, D] ypred = [a, b, c, d]

ys = [A, B, C, D]

ypreds = [a, b, c, d]

ys = [1, -1, -1, 1]

ypreds = [-0.24, -0.32, -0.92, 0.024]

loss = (1 - 0.024) = 0.976] loss contr

loss = (-1 - (-0.92)) = -0.08] loss contr

similarly, loss will be

$\Rightarrow |(A-a)| + |(B-b)| + |(C-c)| + |(D-d)|$

so, instead of doing modulo, we can also do square, as it also converts -ve to +ve.

$$\boxed{\text{loss} = (A-a)^2 + (B-b)^2 + (C-c)^2 + (D-d)^2}$$

this is called MSE loss

so, $\boxed{\text{loss} > 0}$ ✗

Made with Goodnotes

General formula (MSE)

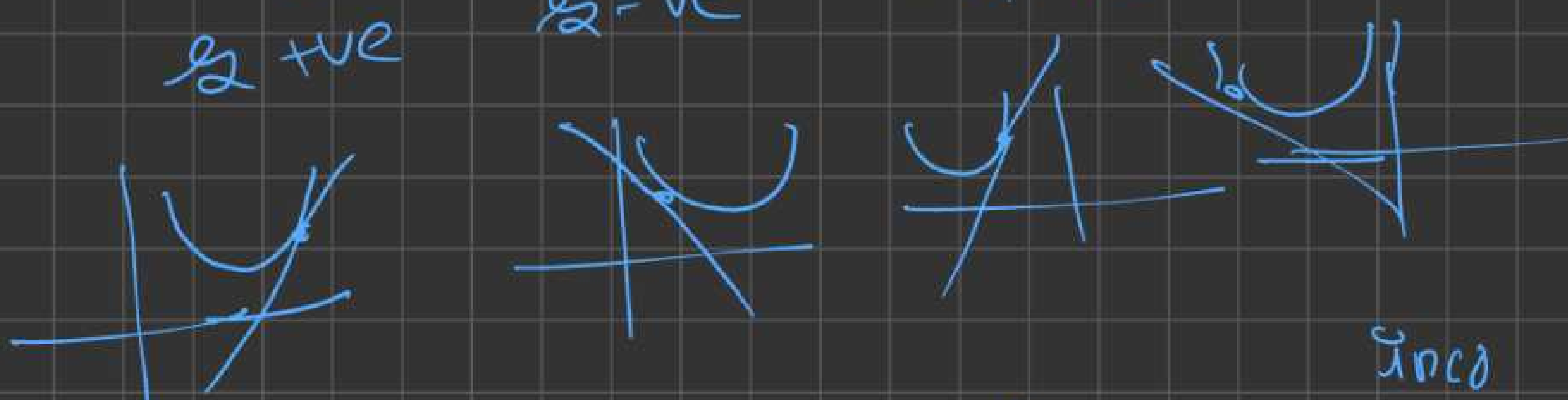
$$MSE = \sum_{i=0}^{n} \left((y(i) - ypred(i)) ** 2 \right)$$

Convention $L = +ve$

w +ve

w -ve

s +ve s -ve s +ve s -ve



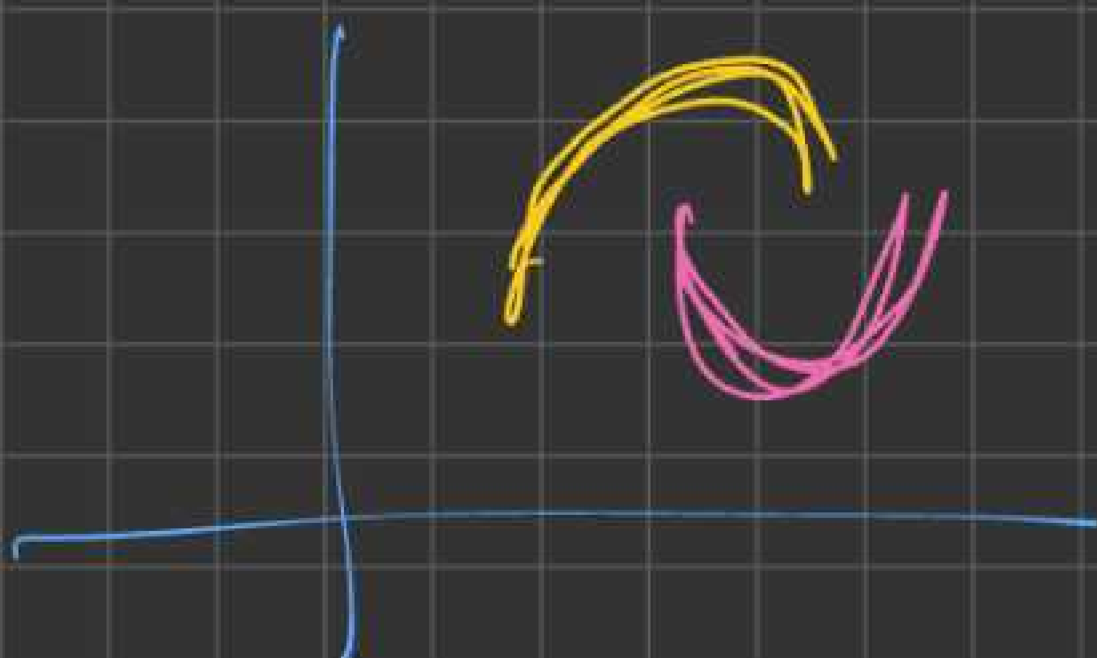
decr (w)   incr. (w)   decr (w)   incr (w)

As per obs., opp. of dirn of slope, adjust your parameters

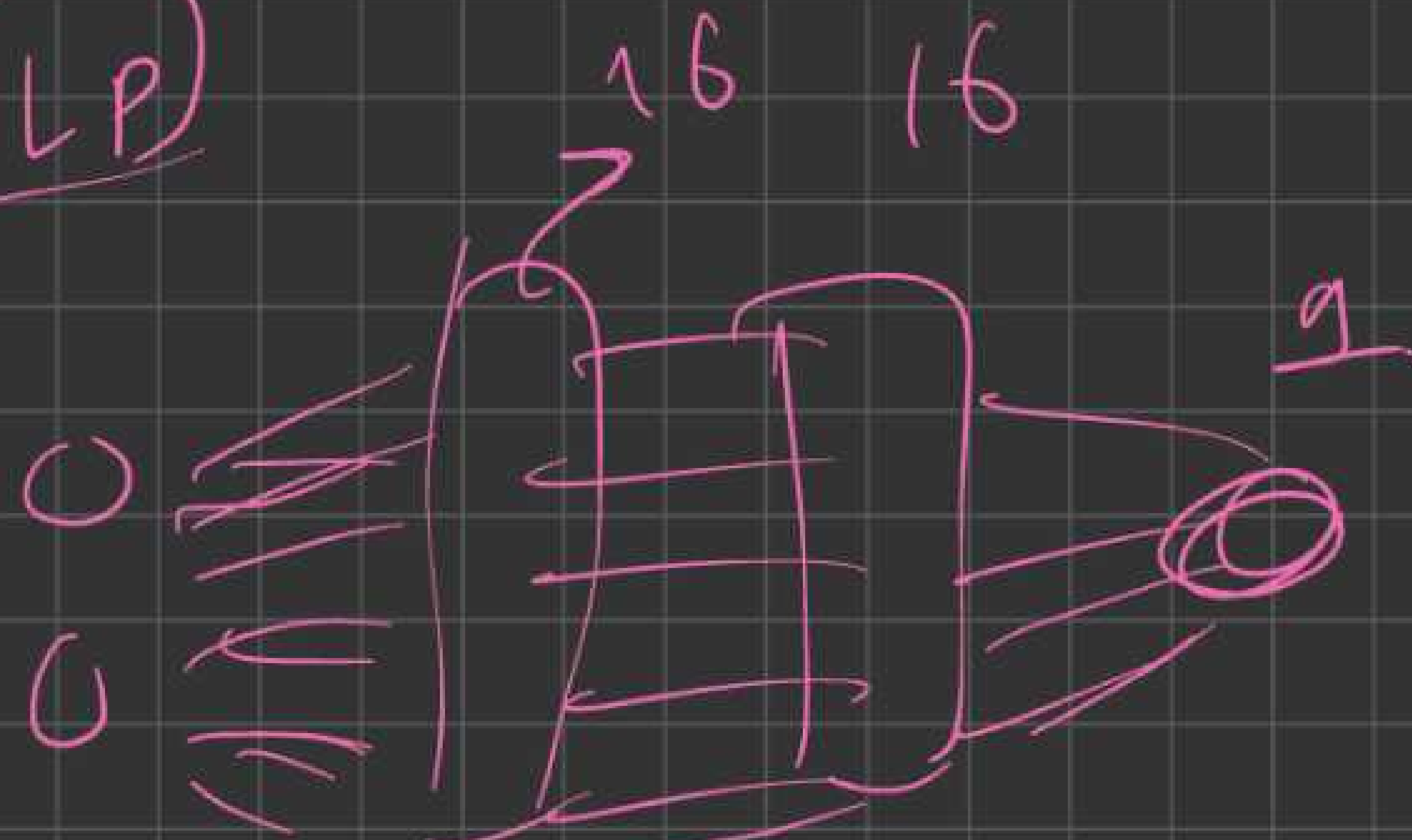
$w.data$ += $(-\alpha) *$ slope ($\rightarrow$ $w.grad$)

Project → Binary classification using a sklearn moon dataset.

moon dataset



MLP



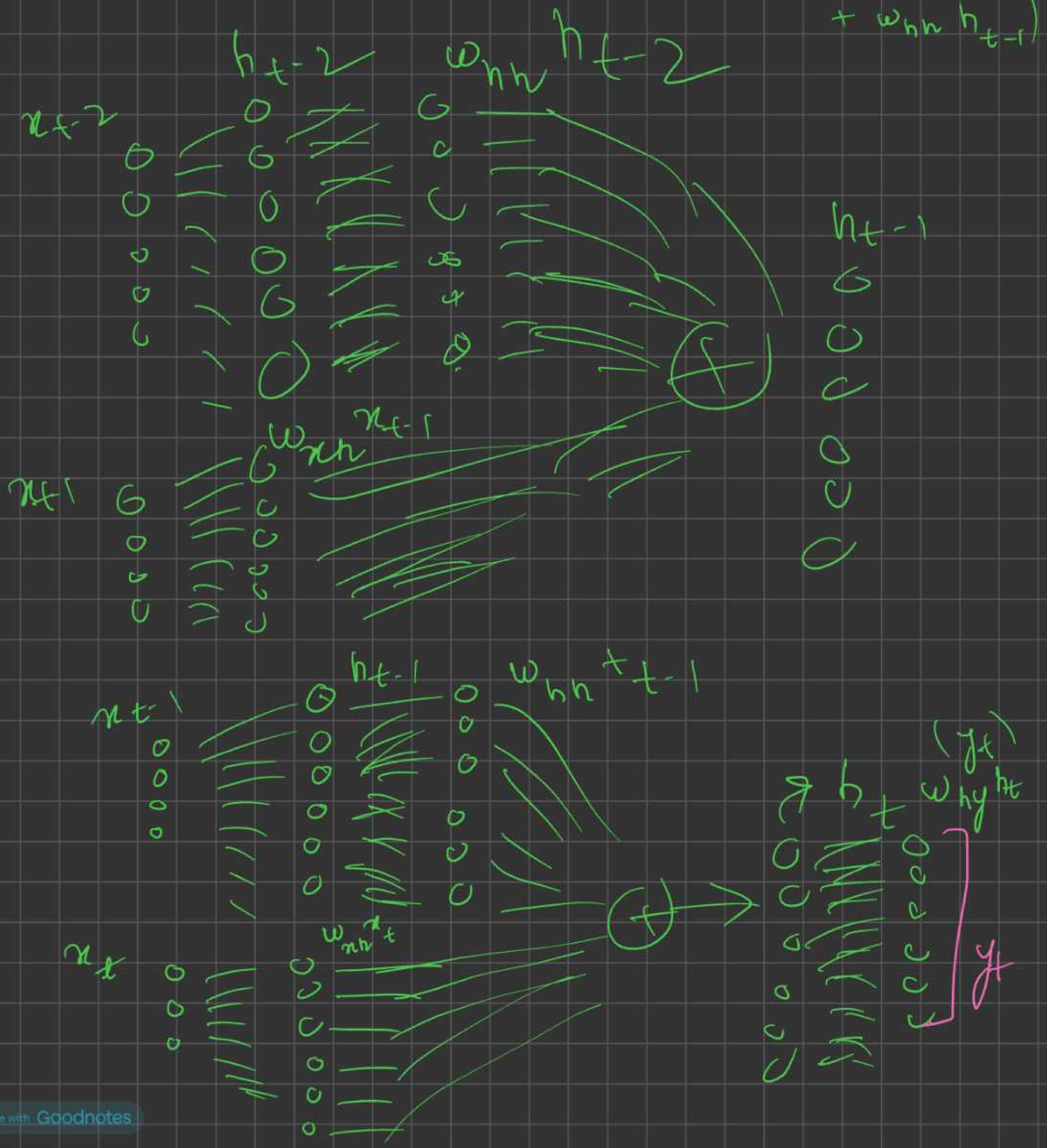
Total parameters $= 2*16 + 16 + 16*16 + 16 + 16*1 + 1 = 337$ params what is epoch?

The No. of times you iterated the entire dataset. Eg:- 100 eg. points, No. of times you iterated over all 100 points??

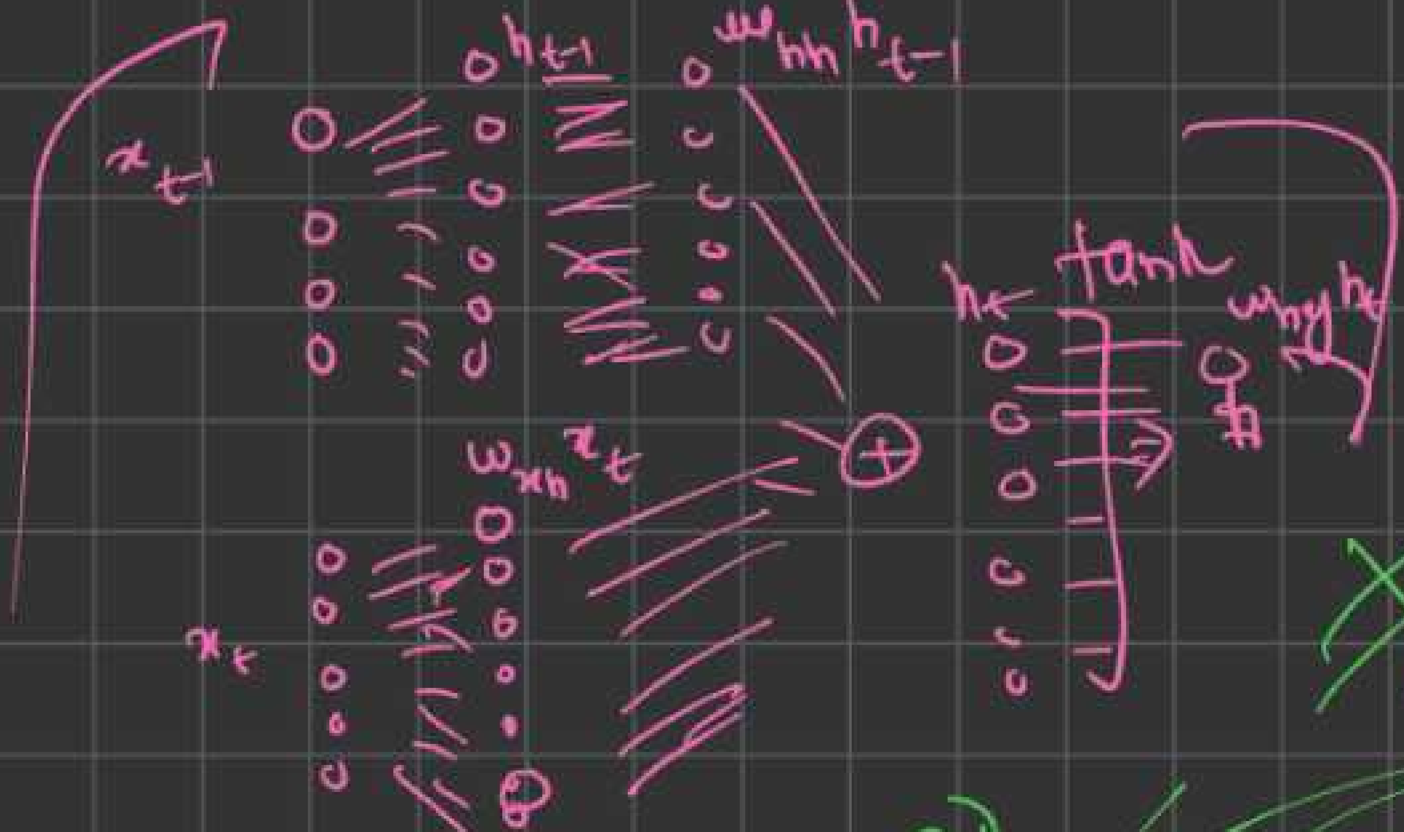
imp pt:- Generally only hidden lay. ers have act. funⁿ but last layer where logits are found, we do not apply act. funⁿ.

$h_{t-1} = (w_{xh} x_{t-1} + w_{hh} h_{t-2})$

$h_t = (w_{xh} x_t + w_{hh} h_{t-1})$



$h_t = \tanh(W_{xh} x_t + W_{hh} h_{t-1})$



$y_t = W_{hy} h_t$

-1.2

-2.1 -1.2

-2.1

-1.1

Supervised

$\frac{1}{1 + e^{-1.2}}$

Softmax, Normalize

-0.36 58.8

$(-1.2,$ 70% 43.39

76.7 30%

51% 56%

43%

correct label RL ?

Policy Grad. ?

$$\frac{e^{-1.2}}{e^{-1.2} + e^{-0.36}}$$

$\log(-1.2)$

e^{-x} . 3

e^x

30%   70%

$(-1.2, -0.36)$

$(-1.2, -0.36)$

30%, 70%

I don't know the label?

Sample action

70%, 30%

Down   up

Suppose we sample, Down.

execute in game ✓

D → D → O → up

Sample action D

gradient = 1.0

Same as of SL?

for D we can just that?

SGD   1

(−1) → +10

reward (+1, −1)

+10.0

ITFLOP +1, −1 → game

+10.0

P

-1.2, -0.36

D

Eg: −

(−1) : (−1) : game

D ↓

stochastic policy

samples actions.

action → Bad outcome

get discourage

action → good outcome

Prev. you know going D good.

wait 250

Okay

D → good

But in RL, we don't know, we need to explore!

Agent takes

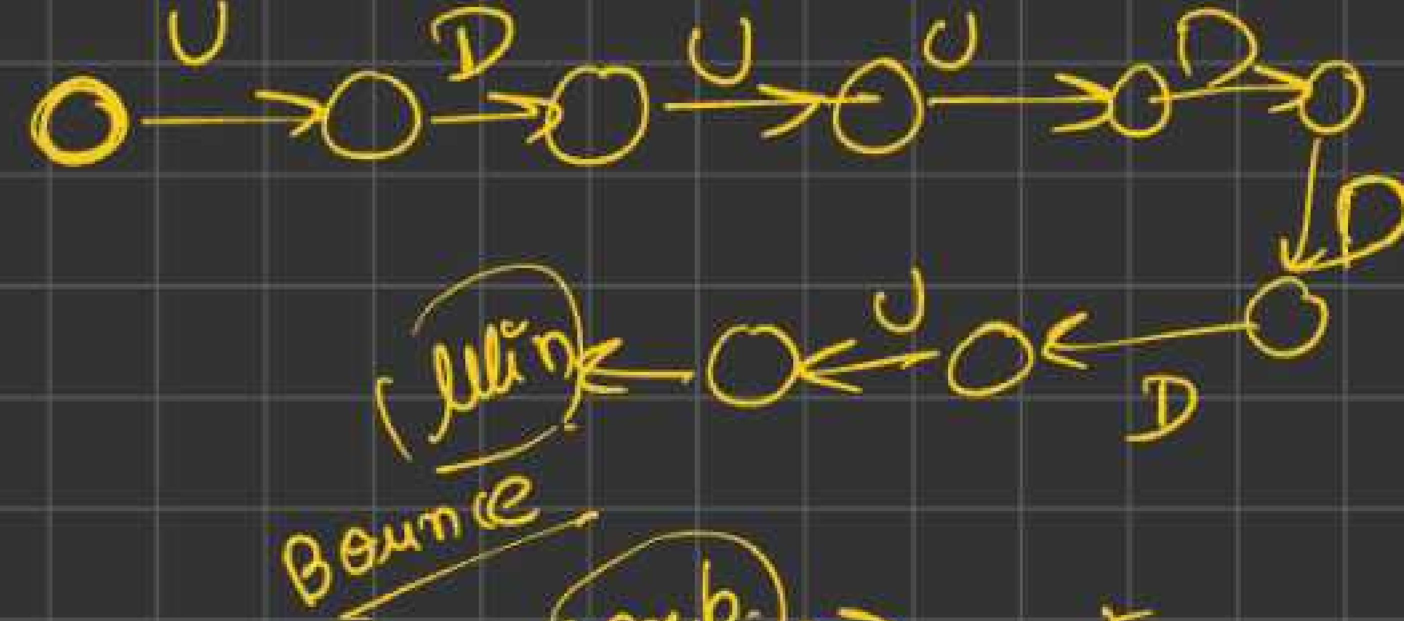
D was good or not?

Pong

reward scalar → end of game? +1, −1

reward. grad. for action

m_1, m_2, 100 games of Pong



1 game = 200 Frames

200 * 100 = 20000

= 20K decision

[change the parameter for U & D

12g won, 88 lost]

88 + 12 = 100g → +ve update +1.0]

200 * 12 = 2400 deci ✓

200 * 88 = 17600 dec.

-ve up.

exp →

$R_t = \sum_{k=0}^{\infty} \gamma^k r_{t+k}$

RMSProp

$R_t = \gamma r_t + \gamma r_{t+1}$

0.98

r_0

$r_0 + 0.99 r_1 + 0.98 r_2$

Batch 10 epi

21 Pong 30

8000 episodes

8000 * 21 episodes

update 160 000

Batch = 10 episodes

Batch = 210 games

1 episode = 21 games

210 x 160 x 3

80 x 80 = 6400]

Numbers

Ph = Policy forw (x)

& sample action] = 2/3

obs, rew, ter, trunc

done = ter or trun.

rs += rew.

Action

52-120

21

(-1, 0, +1)

Reward

rew.

RL → max cumul. reward

21

done true

Episode → 21 game

r1 + r2 + r3 + ... rt

10K steps

episode trunc = true

terminated = true

done = true

truncated → true

env = gym.make(

reward → log →

→ game finish

episode → multiple game]

Score

each game reset.

1.50

51

+1

21

1 episode

cumul.

Made with Goodnotes

1episode → multiple game → 1 game → multiple step (action) → reward

episode No+=1

inputs, hs, act. gp (game level)

re

episode level

episode level

1episode (21 games)

min.
max
41 games

game1 game2 game 3

step step

1epn → hs

200 * 41 = 8200

* 200

16.4 0000

1ep →
(21, 41)
(200*21, 41*200)
(4100, 8200)
Frames per episode

70% UP (0.7)

U (0.3)

(sample) D (-0.7)

0, 0, 0, 0, +1

~~r_t = wor~~

$R_t = r$

$R_t = \boxed{r_t} + \gamma * R_{t-1}$

99%

0 + 0.99 * 0 = 0
0 + 0.99 * 0 = 1
1 + 0.99 * 1 = 0.99
0 + 0.99 * 0.99 = 0.98
0.99 +

(sig (w2 (ReLU (w1 x))))

x = [0, 0, 1, 0, 0, -1, 0, 0, 0, +1]

0, 0, 0, 0, 0, 0, 0

dx = [0, 0, 0, 0, 0, 0, 0, 0, 0]

radd = 0

(x-μ)/σ std normal distrbⁿ

sumadd = 0
radd = 0
radd = r(t) + γ * radd

0 + γ · = (γ)

gam

p = 0.2 (p of going up)
Sample

$p = \text{sigmoid}(z)$

$P(y) \; \boxed{\log(p)}$

$$\frac{\partial \log(p)}{\partial z} = \frac{\partial P(y)}{\partial z} = (y - p)$$

(y = 1/0)

A) y = 1 (sampled UP)

$= (1 - 0.2) = \boxed{0.8} \rightarrow \frac{\partial P(y)}{\partial z} = z.\text{grad}$

AA) You win :- $\boxed{+1}$

$+1 * 0.8 = +0.8$

AB) You loose :- $\boxed{-1} \rightarrow -1 * 0.8 = -0.8$

(0.2) UP → win → so po. of select. UP was low & But then also win, so incr. by factor of 0.8 in z (incr. high)

→ loose → so. p. of UP was low & then you looses also, so network should decr. highly (−0.8)

Sample

B) Down → $\boxed{-0.2}$

BA) win → +1 → −0.2

lose → −1 → +0.2

po. of D was high, & still win, so grad −ve, bcz of po. of max. U, should decr by tiny amt.

bcz of D was low & still loose, possibly by luck, so incr. po. of going UP, but by some amt.